

Article

Fair Max–Min Diversity Maximization in Streaming and Sliding-Window Models [†]

Yanhao Wang ¹ , Francesco Fabbri ², Michael Mathioudakis ^{3,*}  and Jia Li ¹

¹ School of Data Science and Engineering, East China Normal University, Shanghai 200062, China

² Spotify, 08000 Barcelona, Spain

³ Department of Computer Science, University of Helsinki, 00560 Helsinki, Finland

* Correspondence: michael.mathioudakis@helsinki.fi

[†] This paper is an extended version of our paper published in Proceedings of the IEEE 38th International Conference on Data Engineering (ICDE 2022); pp. 41–53.

Abstract: Diversity maximization is a fundamental problem with broad applications in data summarization, web search, and recommender systems. Given a set X of n elements, the problem asks for a subset S of $k \ll n$ elements with maximum diversity, as quantified by the dissimilarities among the elements in S . In this paper, we study diversity maximization with fairness constraints in streaming and sliding-window models. Specifically, we focus on the max–min diversity maximization problem, which selects a subset S that maximizes the minimum distance (dissimilarity) between any pair of distinct elements within it. Assuming that the set X is partitioned into m disjoint groups by a specific sensitive attribute, e.g., sex or race, ensuring fairness requires that the selected subset S contains k_i elements from each group $i \in [m]$. Although diversity maximization has been extensively studied, existing algorithms for fair max–min diversity maximization are inefficient for data streams. To address the problem, we first design efficient approximation algorithms for this problem in the (insert-only) streaming model, where data arrive one element at a time, and a solution should be computed based on the elements observed in one pass. Furthermore, we propose approximation algorithms for this problem in the sliding-window model, where only the latest w elements in the stream are considered for computation to capture the recency of the data. Experimental results on real-world and synthetic datasets show that our algorithms provide solutions of comparable quality to the state-of-the-art offline algorithms while running several orders of magnitude faster in the streaming and sliding-window settings.

Keywords: diversity maximization; group fairness; streaming algorithm; sliding-window algorithm



Citation: Wang, Y.; Fabbri, F.; Mathioudakis, M.; Li, J. Fair Max–Min Diversity Maximization in Streaming and Sliding-Window Models. *Entropy* **2023**, *25*, 1066. <https://doi.org/10.3390/e25071066>

Academic Editor: Jaesung Lee

Received: 19 June 2023

Revised: 12 July 2023

Accepted: 13 July 2023

Published: 14 July 2023



Copyright: © 2023 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Data summarization is a common approach to tackling the challenges of a large volume of data in data-intensive applications. That is because, rather than performing high-complexity analyses on the whole dataset, it is often beneficial to perform them on a representative and significantly smaller summary of the dataset, thus reducing the processing costs in terms of both running time and space usage. Typical techniques for data summarization [1] include sampling, sketching, coresets, and diverse data selection.

In this paper, we focus on diversity-aware data summarization, which finds application in a wide range of real-world problems. For example, in database query processing [2,3], web search [4,5], and recommender systems [6], the output might be too large to be presented to the user in its entirety, even after filtering the results by relevance. One feasible solution, then, is to present the user with a small but diverse subset that is easy to process and representative of the complete results. As another example, when training machine learning models on massive data, feature and subset selection is a standard method to improve efficiency. As indicated by [7,8], selecting diverse features or subsets can lead to

a better balance between efficiency and accuracy. A key technical problem in such cases is diversity maximization [9–20].

In more detail, for a given set X of elements in some metric space and a size constraint k , diversity maximization asks for a subset of k elements with maximum diversity. Formally, diversity is quantified by a function that captures how well a subset spans the range of elements in X , and is typically defined in terms of distances or dissimilarities among elements in the subset. Prior studies [3,4,6,12] have suggested many different objectives of this kind. Two of the most popular ones are max-sum dispersion, which aims to maximize the sum of the distances between all pairs of elements in the selected subset S , and max-min dispersion, which aims to maximize the minimum distance between any pair of distinct elements in S . Figure 1 illustrates a selection of the 10 most diverse points from a two-dimensional point set with each of the two objectives for diversity maximization. As shown in Figure 1, max-sum dispersion tends to select “outliers” and may include highly similar elements in the solution, making it unsuitable for applications requiring more uniform coverage of the span of data. Therefore, we focus on diversity maximization with the objective of max-min dispersion, referred to as max-min diversity maximization, in this paper.

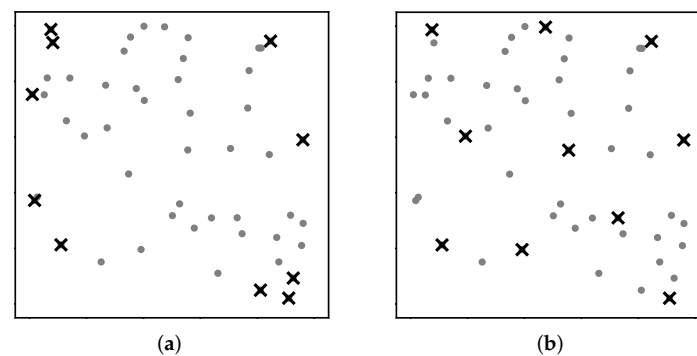


Figure 1. Comparison of (a) max-sum dispersion (MSD) and (b) max-min dispersion (MMD) for diversity maximization on a dataset of one hundred points. We use circles and crossmarks to denote all points in the dataset and the points selected based on MSD and MMD.

In addition to diversity, fairness in data summarization is also attracting increasing attention [8,21–27]. Several studies reveal that the biases with respect to (w.r.t.) sensitive attributes, such as sex, race, or age, in underlying datasets can be retained in the summaries and could lead to unfairness in data-driven social computational systems such as education, recruitment, and banking [8,23,26]. One of the most common notions for fairness in data summarization is group fairness [8,21–23,27], which partitions the dataset into m disjoint groups based on a specific sensitive attribute and introduces a fairness constraint that limits the number of elements from group i in the data summary to k_i for every group $i \in [m]$ (see Figure 2 for an illustrative example). However, most existing methods for diversity maximization cannot easily be adapted to satisfy such fairness constraints. Moreover, a few methods that can deal with fairness constraints are specific to max-sum diversity maximization [9,11,13]. To the best of our knowledge, the methods in [17,20] are the only means of max-min diversity maximization with fairness constraints.

Furthermore, since many applications of diversity maximization are in the realm of massive data analysis, it is essential to design efficient algorithms for processing large-scale datasets. The (insert-only) streaming and sliding-window models are well-recognized frameworks for big data processing. In the streaming model, an algorithm is only permitted to process each element in the dataset sequentially in one pass, is allowed to take time and space that are sublinear to or even independent of the dataset size, and is required to provide solutions of comparable quality to those returned by the offline algorithms. In the sliding-window model, the computation is further restricted to the latest w elements

in the stream, and an algorithm is required to find good solutions in sublinear time and space w.r.t. the window size. However, the only known algorithms [17,20] for fair max–min diversity maximization are designed for the offline setting and are very inefficient in the streaming and sliding-window models.

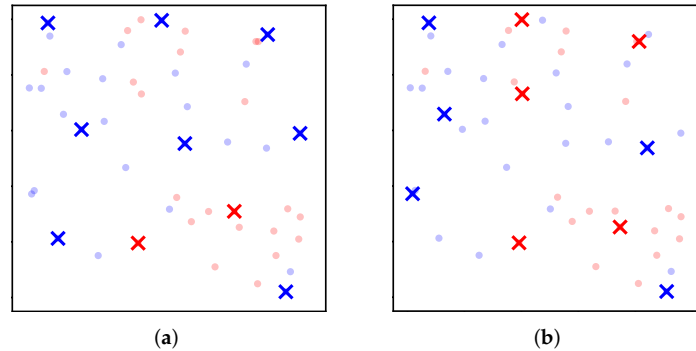


Figure 2. Comparison of (a) unconstrained max–min diversity maximization and (b) fair max–min diversity maximization. We have a set of individuals, each described by two attributes, partitioned into two disjoint groups of red and blue, respectively. Fair diversity maximization returns a subset of size 10 that maximizes diversity in terms of attributes and contains an equal number (i.e., $k_i = 5$) of elements from both groups.

Our Contributions: In this paper, we propose novel streaming and sliding-window algorithms for the max–min diversity maximization problem with fairness constraints. Our main contributions are summarized as follows:

- We formally define the problem of fair max–min diversity maximization (FDM) in metric spaces. Then, we describe the existing streaming and sliding-window algorithms for (unconstrained) max–min diversity maximization [14]. In particular, we improve the approximation ratio of the existing streaming algorithm from $\frac{1-\varepsilon}{5}$ to $\frac{1-\varepsilon}{2}$ for any parameter $\varepsilon \in (0, 1)$ by refining the analysis of [14].
- We propose two novel streaming algorithms for FDM. Our first algorithm, called SFDM1, is $\frac{1-\varepsilon}{4}$ -approximate for FDM when there are two groups in the dataset. It takes $O(\frac{k \log \Delta}{\varepsilon})$ time per element in the streaming processing, where Δ is the ratio of the maximum and minimum distances between any pair of elements, spends $O(\frac{k^2 \log \Delta}{\varepsilon})$ time for post-processing, and stores $O(\frac{k \log \Delta}{\varepsilon})$ elements in memory. Our second algorithm, called SFDM2, is $\frac{1-\varepsilon}{3m+2}$ -approximate for FDM with an arbitrary number m of groups. SFDM2 also takes $O(\frac{k \log \Delta}{\varepsilon})$ time per element in the streaming processing but requires a longer $O(\frac{k^2 m \log \Delta}{\varepsilon} \cdot (m + \log^2 k))$ time for post-processing and stores $O(\frac{km \log \Delta}{\varepsilon})$ elements in memory.
- We further extend our two streaming algorithms to the sliding-window model. The extended SWFDM1 and SWFDM2 algorithms achieve approximation factors of $\Theta(1)$ and $\Theta(m^{-1})$ for FDM with $m = 2$ and an arbitrary m when any $\Theta(1)$ -approximation algorithm for unconstrained max–min diversity maximization is used for post-processing. Additionally, their time and space complexities increase by a factor of $O(\frac{\log \Delta}{\varepsilon})$ compared with SFDM1 and SFDM2, respectively.
- Finally, we evaluate the performance of our proposed algorithms against the state-of-the-art algorithms on several real-world and synthetic datasets. The results demonstrate that our algorithms provide solutions of comparable quality for FDM to those returned by the state-of-the-art algorithms while running several orders of magnitude faster in the streaming and sliding-window settings.

A preliminary version of this paper is published in [28]. In this extended version, we make the following novel contributions with respect to [28]: (1) We propose two

novel algorithms for FDM in the sliding-window model along with the implementation of an existing algorithm for unconstrained max–min diversity maximization in the sliding-window model [14]. Moreover, we analyze the approximation factors and complexities of the two algorithms for fair sliding-window diversity maximization; (2) We conduct more comprehensive examinations of our streaming algorithms by implementing and comparing them with a new offline baseline called FairGreedyFlow [20], which achieves a better approximation factor than previous offline algorithms. The additional results further confirm the superior performance of our streaming algorithms; (3) We conduct new experiments for FDM in the sliding-window setting to evaluate the performance of our sliding-window algorithms compared with the existing offline algorithms. The new experimental results validate their efficiency, effectiveness, and scalability.

Paper Organization: The rest of this paper is organized as follows. The related work is reviewed in Section 2. In Section 3, we introduce the basic concepts and formally define the FDM problem. In Section 4, we first propose our streaming algorithms for FDM. In Section 5, we further design our sliding-window algorithms for FDM. Our experimental setup and results are described in Section 6. Finally, we conclude the paper in Section 7.

2. Related Work

Diversity maximization has been extensively studied over the last two decades. Existing studies mainly focus on two popular objectives—i.e., max–sum dispersion [11–16,29–31] and max–min dispersion [12,14,16–18,20,31], and their variants [12,32].

An early study [33] proved that both max–sum and max–min diversity maximization problems are NP-hard even in metric spaces. The classic approaches to both problems are the greedy algorithms [34,35], which achieves the best possible approximation ratio of $\frac{1}{2}$ unless $P = NP$. Indyk et al. [12] proposed composable coresets-based approximation algorithms for diversity maximization. Aghamolaei et al. [31] improved the approximation ratios in [12]. Ceccarello et al. [16] proposed coresets-based approximation algorithms for diversity maximization in MapReduce and streaming settings where the metric space has a bounded doubling dimension. Borassi et al. [14] proposed sliding-window algorithms for diversity maximization. Epasto et al. [36] further proposed improved sliding-window algorithms for diversity maximization specific to the Euclidean space. Drosou and Pitoura [18] studied max–min diversity maximization on dynamic data. They proposed a $\frac{b-1}{2b^2}$ -approximation algorithm using a cover tree of base b . Bauckhage et al. [15] proposed an adiabatic quantum computing solution for max–sum diversification. Zhang and Gionis [19] extended diversity maximization to clustered data. Nevertheless, all the above methods only consider diversity maximization problems without fairness constraints.

There were several studies on diversity maximization under matroid constraints, of which the fairness constraints are special cases. Abbassi et al. [11] proposed a $(\frac{1}{2} - \epsilon)$ -approximation local search algorithm for max–sum diversification under matroid constraints. Borodin et al. [9] proposed a $(\frac{1}{2} - \epsilon)$ -approximation algorithm for maximizing the sum of a submodular function and a max–sum dispersion function. Cevallos et al. [30] extended the local search algorithm for distances of a negative type. They also proposed a PTAS for this problem via convex programming [29]. Bhaskara et al. [37] proposed a $\frac{1}{8}$ -approximation algorithm for sum–min diversity maximization under matroid constraints using linear relaxations. Ceccarello et al. [13] proposed a coresets-based approach to matroid-constrained max–sum diversification in metric spaces of bounded doubling dimension. Nevertheless, the above methods are still not applicable to the max–min dispersion problem. The only known algorithms for fair max–min diversity maximization in [17,20,38] are offline algorithms and inefficient for data streams. We will compare our proposed algorithms with these, both theoretically and empirically. To the best of our knowledge, there has not been any previous streaming or sliding-window algorithm for fair max–min diversity maximization.

In addition to diversity maximization, fairness has also been considered in many other data summarization problems, such as k -center [21–23], determinantal point processes [8],

coresets for k -means clustering [24,25], and submodular maximization [26,27]. However, since their optimization objectives differ from diversity maximization, the proposed algorithms for their fair variants cannot be directly used for our problem.

3. Preliminaries

In this section, we introduce the basic concepts and formally define the fair max–min diversity maximization problem.

Let X be a set of n elements from a metric space with distance function $d(\cdot, \cdot)$ capturing the dissimilarities among elements. Recall that $d(\cdot, \cdot)$ is nonnegative, symmetric, and satisfies the triangle inequality—i.e., $d(x, y) + d(y, z) \geq d(x, z)$ for any $x, y, z \in X$. Note that all the algorithms and analyses in this paper are general for any distance metric. We further generalize the notion of distance to an element x and a set S as the distance between x and its nearest neighbor in S —i.e., $d(x, S) = \min_{y \in S} d(x, y)$.

Our focus in this paper is to find a small subset of most diverse elements from X . Given a subset $S \subseteq X$, its diversity $\text{div}(S)$ is defined as the minimum of the pairwise distances between any two distinct elements in S , i.e., $\text{div}(S) = \min_{x, y \in S, x \neq y} d(x, y)$. The unconstrained version of diversity maximization (DM) asks for a subset $S \subseteq X$ of k elements maximizing $\text{div}(S)$, i.e., $S^* = \arg \max_{S \subseteq X: |S|=k} \text{div}(S)$. We use $\text{OPT} = \text{div}(S^*)$ to denote the diversity of the optimal solution S^* for DM. This problem has been proven to be NP-complete [33], and no polynomial-time algorithm can achieve an approximation factor that is better than $\frac{1}{2}$ unless $P=NP$. One approach to DM is the $\frac{1}{2}$ -approximation greedy algorithm [34,39] (known as GMM) in the offline setting.

We introduce fairness to diversity maximization when X is composed of several demographic groups defined by a certain sensitive attribute, e.g., sex or race. Formally, suppose that X is divided into m disjoint groups $[1, \dots, m]$ ($[m]$ for short) and a function $c: X \rightarrow [m]$ maps each element $x \in X$ to its group. Let $X_i = \{x \in X : c(x) = i\}$ be the subset of elements from group i in X . Obviously, we have $\bigcup_{i=1}^m X_i = X$ and $X_i \cap X_j = \emptyset$ for any $i \neq j$. The fairness constraint assigns a positive integer k_i to each of the m groups and restricts the number of elements from group i in the solution to k_i . We assume that $\sum_{i=1}^m k_i = k$. The fair max-min diversity maximization problem is defined as:

Definition 1 (FDM). Given a set X of n elements with $X = \bigcup_{i=1}^m X_i$ and m size constraints $k_1, \dots, k_m \in \mathbb{Z}^+$, find a subset S that contains k_i elements from X_i and maximizes $\text{div}(S)$ —i.e., $S_f^* = \arg \max_{S \subseteq X: |S \cap X_i| = k_i, \forall i \in [m]} \text{div}(S)$.

We use $\text{OPT}_f = \text{div}(S_f^*)$ to denote the diversity of the optimal solution S_f^* for FDM. Since DM is a special case of FDM when $m = 1$, FDM is NP-hard up to a $\frac{1}{2}$ -approximation. In addition, our FDM problem is closely related to the concept of matroid [40] in combinatorics. Given a ground set V , a matroid is a pair $\mathcal{M} = (V, \mathcal{I})$ where $\mathcal{I}$ is a family of subsets of V (called independent sets) with the following properties: (i) $\emptyset \in \mathcal{I}$; (ii) for each $A \subseteq B \subseteq V$, if $B \in \mathcal{I}$ then $A \in \mathcal{I}$ (hereditary); and (iii) if $A \in \mathcal{I}$, $B \in \mathcal{I}$, and $|A| > |B|$, then there exists $x \in A \setminus B$ such that $B \cup \{x\} \in \mathcal{I}$ (augmentation). An independent set is maximal if it is not a proper subset of any other independent set. A basic property of $\mathcal{M}$ is that all its maximal independent sets have the same size, denoted as the matroid's rank. As is easy to verify, our fairness constraint is a case of rank- k partition matroids, where the ground set is partitioned into disjoint groups and the independent sets are exactly the sets in which, for each group, the number of elements from this group is, at most, the group capacity. Our algorithms for general m in Sections 4 and 5 will be built on matroids.

In this paper, we first consider FDM in the streaming setting, where the elements in X arrive one at a time. Here, we use $t(x)$ to denote the time when an element x is observed and $X^{(T)} = \{x \in X : t(x) \leq T\}$ to denote the subset of elements observed from X until time T . A streaming algorithm should process each element sequentially in one pass using limited space (typically independent of n) and return a valid approximate solution S (if it exists) for FDM on $X^{(T)}$ at any time T . We further study FDM in the sliding-window

setting, where the window $W^{(T)}$ always contains the last w elements observed from X until time T , i.e., $W^{(T)} = \{x \in X : T - w + 1 \leq t(x) \leq T\}$. A sliding-window algorithm should provide a valid approximate solution S (if it exists) for FDM on $W^{(T)}$ at any time T .

4. Streaming Algorithms

As has been shown in Section 3, FDM is NP-hard. Thus, we focus on efficient approximation algorithms for FDM. In this section, we first describe the existing algorithms for unconstrained diversity maximization in the streaming model on which our streaming algorithms will be built. We then propose a $\frac{1-\varepsilon}{4}$ -approximation streaming algorithm for FDM in the special case that there are only two groups in the dataset. Finally, we propose a $\frac{1-\varepsilon}{3m+2}$ -approximation streaming algorithm for FDM on a dataset with an arbitrary number m of groups.

4.1. (Unconstrained) Streaming Algorithm

We first present the streaming algorithm of [14] for (unconstrained) diversity maximization in Algorithm 1. Let $d_{\min} = \min_{x,y \in X, x \neq y} d(x,y)$, $d_{\max} = \max_{x,y \in X, x \neq y} d(x,y)$ and $\Delta = \frac{d_{\max}}{d_{\min}}$. Obviously, it always holds that $\text{OPT} \in [d_{\min}, d_{\max}]$. First, it maintains a sequence $\mathcal{U}$ of values for guessing OPT within a relative error of $1 - \varepsilon$ and initializes an empty solution S_μ for each $\mu \in \mathcal{U}$ before processing the stream (Lines 1 and 2). Then, for each $x \in X$ and each $\mu \in \mathcal{U}$, if S_μ contains less than k elements and the distance between x and S_μ is at least μ , it will add x to S_μ (Lines 3–6). After processing all elements in X , the candidate solution that contains k elements and maximizes the diversity is returned as the solution S for DM (Line 7). Algorithm 1 is proven to be a $\frac{1-\varepsilon}{5}$ -approximation algorithm for max–min diversity maximization [14]. In Theorem 1, its approximation ratio is improved to $\frac{1-\varepsilon}{2}$ by refining the analysis of [14].

Algorithm 1 SDM

Input: Stream X , distance metric $d(\cdot, \cdot)$, parameter $\varepsilon \in (0, 1)$, solution size $k \in \mathbb{Z}^+$

Output: A set $S \subseteq X$ with $|S| = k$

- 1: $\mathcal{U} = \{\frac{d_{\min}}{(1-\varepsilon)^j} : j \in \mathbb{Z}_0^+ \wedge (1-\varepsilon)^j \geq \frac{d_{\min}}{d_{\max}}\}$
 - 2: Initialize $S_\mu = \emptyset$ for each $\mu \in \mathcal{U}$
 - 3: **for all** $x \in X$ **do**
 - 4: **for all** $\mu \in \mathcal{U}$ **do**
 - 5: **if** $|S_\mu| < k$ and $d(x, S_\mu) \geq \mu$ **then**
 - 6: $S_\mu \leftarrow S_\mu \cup \{x\}$
 - 7: **return** $S \leftarrow \arg \max_{\mu \in \mathcal{U}: |S_\mu|=k} \text{div}(S_\mu)$
-

Theorem 1. Algorithm 1 is a $\frac{1-\varepsilon}{2}$ -approximation algorithm for max–min diversity maximization.

Proof. For each $\mu \in \mathcal{U}$, there are two cases for S_μ after processing all elements in X : (1) If $|S_\mu| = k$, the condition of Line 5 guarantees that $\text{div}(S_\mu) \geq \mu$; (2) If $|S_\mu| < k$, it holds that $d(x, S_\mu) < \mu$ for every $x \in X \setminus S_\mu$ since the fact that x is not added to S_μ implies that $d(x, S_\mu) < \mu$, as $|S_\mu| < k$. Let us consider a candidate solution S_μ with $|S_\mu| < k$. Suppose that $S^* = \{s_1^*, \dots, s_k^*\}$ is the optimal solution for DM on X . We define a function $f: S^* \rightarrow S_\mu$ that maps each element in S^* to its nearest neighbor in S_μ . As is shown above, $d(s^*, f(s^*)) < \mu$ for each $s^* \in S^*$. Because $|S_\mu| < k$ and $|S^*| = k$, two distinct elements $s_a^*, s_b^* \in S^*$ with $f(s_a^*) = f(s_b^*)$ must exist. For such s_a^*, s_b^* , we have

$$d(s_a^*, s_b^*) \leq d(s_a^*, f(s_a^*)) + d(s_b^*, f(s_b^*)) < 2\mu$$

according to the triangle inequality. Thus, $\text{OPT} = \text{div}(S^*) \leq d(s_a^*, s_b^*) < 2\mu$ if $|S_\mu| < k$. Let μ' be the smallest $\mu \in \mathcal{U}$ with $|S_\mu| < k$. We obtained $\text{div}(S^*) < 2\mu'$ from the above results.

Additionally, for $\mu'' = (1 - \varepsilon)\mu'$, we must have $|S_{\mu''}| = k$ and $\text{div}(S_{\mu''}) \geq \mu''$. Therefore, we have $\text{div}(S) \geq \mu'' = (1 - \varepsilon)\mu' \geq \frac{1-\varepsilon}{2} \cdot \text{div}(S^*)$. $\square$

In terms of complexity, Algorithm 1 stores $O(\frac{k \log \Delta}{\varepsilon})$ elements and takes $O(\frac{k \log \Delta}{\varepsilon})$ time per element, since it makes $O(\frac{\log \Delta}{\varepsilon})$ guesses for OPT, keeps, at most, k elements in each candidate and requires, at most, k distance computations to decide whether to add an element to a candidate.

4.2. Fair Streaming Algorithm for $m = 2$

The procedure of our streaming algorithm in case of $m = 2$, called SFDM1, is described in Algorithm 2 and illustrated in Figure 3. In general, the algorithm runs in two phases: stream processing and post-processing. In the stream processing (Lines 1–6), for each guess $\mu \in \mathcal{U}$ of OPT_f , it utilizes Algorithm 1 to keep a group-blind candidate S_μ with size constraint k and two group-specific candidates $S_{\mu,1}$ and $S_{\mu,2}$ with size constraints k_1 and k_2 for X_1 and X_2 , respectively. The only difference from Algorithm 1 is that the elements are filtered by group to maintain $S_{\mu,1}$ and $S_{\mu,2}$. After processing all elements of X in one pass, it will post-process the group-blind candidates to make them satisfy the fairness constraint (Lines 7–15). The post-processing is only performed on a subset $\mathcal{U}'$ of $\mathcal{U}$, where S_μ contains k elements and $S_{\mu,i}$ contains k_i elements for each group $i \in \{1, 2\}$. For each $\mu \in \mathcal{U}'$, S_μ , either has satisfied the fairness constraint or has one over-filled group i_o and another under-filled group i_u . If S_μ is not yet a fair solution, S_μ will be balanced for fairness by first adding $k_{i_u} - k'_{i_u}$ elements, where $k'_{i_u} = |S_\mu \cap X_{i_u}|$, from S_{μ,i_u} to S_μ , and then removing the same number of elements from $S_\mu \cap X_{i_o}$. The elements to be added and removed are selected greedily, as in GMM [39], to minimize the loss in diversity: the element in S_{μ,i_u} that is the furthest from $S_\mu \cap X_{i_u}$ is picked for each insertion; and the element in $S_\mu \cap X_{i_o}$ that is the closest to $S_\mu \cap X_{i_u}$ is picked for each deletion. Finally, the fair candidate with the maximum diversity after post-processing is returned as the final solution for FDM (Line 16). Next, we will theoretically analyze the approximation ratio and complexity of SFDM1.

Algorithm 2 SFDM1

Input: Stream $X = X_1 \cup X_2$, distance metric $d(\cdot, \cdot)$, parameter $\varepsilon \in (0, 1)$, size constraints $k_1, k_2 \in \mathbb{Z}^+$ ($k = k_1 + k_2$)

Output: A set $S \subseteq X$ s.t. $|S \cap X_i| = k_i$ for $i \in \{1, 2\}$

```

▷ Stream processing
1:  $\mathcal{U} = \{\frac{d_{\min}}{(1-\varepsilon)^j} : j \in \mathbb{Z}_0^+ \wedge (1-\varepsilon)^j \geq \frac{d_{\min}}{d_{\max}}\}$ 
2: Initialize  $S_\mu, S_{\mu,i} = \emptyset$  for every  $\mu \in \mathcal{U}$  and  $i \in \{1, 2\}$ 
3: for all  $x \in X$  do
4:   Run Lines 3–6 of Algorithm 1 to update  $S_\mu$  w.r.t.  $x$ 
5:   if  $c(x) = i$  then
6:     Run Lines 3–6 of Algorithm 1 to update  $S_{\mu,i}$  w.r.t.  $x$  with size constraint  $k_i$ 

▷ Post-processing
7:  $\mathcal{U}' = \{\mu \in \mathcal{U} : |S_\mu| = k \wedge |S_{\mu,i}| = k_i, \forall i \in \{1, 2\}\}$ 
8: for all  $\mu \in \mathcal{U}'$  do
9:   if  $|S_\mu \cap X_i| < k_i$  for some  $i \in \{1, 2\}$  then
10:    while  $|S_\mu \cap X_i| < k_i$  do
11:       $x^+ \leftarrow \arg \max_{x \in S_{\mu,i}} d(x, S_\mu \cap X_i)$ 
12:       $S_\mu \leftarrow S_\mu \cup \{x^+\}$ 
13:    while  $|S_\mu| > k$  do
14:       $x^- \leftarrow \arg \min_{x \in S_\mu \setminus X_i} d(x, S_\mu \cap X_i)$ 
15:       $S_\mu \leftarrow S_\mu \setminus \{x^-\}$ 
16: return  $S \leftarrow \arg \max_{\mu \in \mathcal{U}'} \text{div}(S_\mu)$ 

```

Theoretical Analysis: We prove that SFDM1 achieves an approximation ratio of $\frac{1-\varepsilon}{4}$ for FDM, where $\varepsilon \in (0, 1)$, in Theorem 2. The proof is based on (i) the existence of $\mu' \in \mathcal{U}'$ such that $\mu' \geq \frac{1-\varepsilon}{2} \cdot \text{OPT}_f$ (Lemma 1) and (ii) $\text{div}(S_\mu) \geq \frac{\mu}{2}$ for each $\mu \in \mathcal{U}'$ after post-processing (Lemma 2). Then, we analyze the complexity of SFDM1 in Theorem 3.

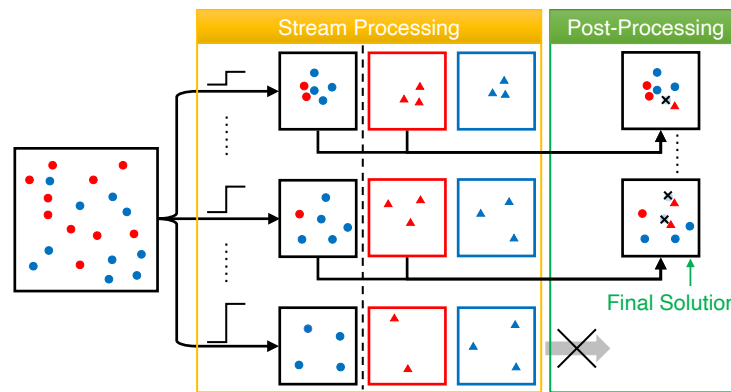


Figure 3. Illustration of the SFDM1 algorithm. During stream processing, one group-blind and two group-specific candidates are maintained for each guess μ of OPT_f . Then, a subset of group-blind candidates is selected for post-processing by adding the elements from the under-filled group before deleting the elements from the over-filled one.

Lemma 1. Let μ' be the largest $\mu \in \mathcal{U}'$. It holds that $\mu' \geq \frac{1-\varepsilon}{2} \cdot \text{OPT}_f$, where OPT_f is the optimal diversity of FDM on X .

Proof. First of all, we have $\text{OPT}_f \leq \text{OPT}$, where OPT is the optimal diversity of unconstrained DM with $k = k_1 + k_2$ on X , since any valid solution for FDM must also be a valid solution for DM. Moreover, it holds that $\text{OPT}_f \leq \text{OPT}_{k_i}$, where OPT_{k_i} is the optimal diversity of unconstrained DM with size constraint k_i on X_i for both $i \in \{1, 2\}$, because the optimal solution must contain k_i elements from X_i and $\text{div}(\cdot)$ is a monotonically non-increasing function—i.e., $\text{div}(S \cup \{x\}) \leq \text{div}(S)$ for any $S \subseteq X$ and $x \in S \setminus X$. Therefore, we prove that $\text{OPT}_f \leq \text{div}(S^* \cap X_i) \leq \text{OPT}_{k_i}$.

Then, according to the results of Theorem 1, we have $\text{OPT} < 2\mu$ if $S_\mu < k$ and $\text{OPT}_{k_i} < 2\mu$ if $|S_{\mu,i}| < k_i$ for each $i \in \{1, 2\}$. Note that μ' is the largest $\mu \in \mathcal{U}$, such that $|S_\mu| = k$, $|S_{\mu,1}| = k_1$, and $|S_{\mu,2}| = k_2$ after stream processing. For $\mu'' = \frac{\mu'}{1-\varepsilon} \in \mathcal{U}$, we have either $|S_{\mu''}| < k$ or $|S_{\mu'',i}| < k_i$ for some $i \in \{1, 2\}$. Therefore, it holds that $\text{OPT}_f < 2\mu'' \leq \frac{2}{1-\varepsilon} \cdot \mu'$ and we conclude the proof. $\square$

Lemma 2. For each $\mu \in \mathcal{U}'$, the candidate solution S_μ must satisfy $\text{div}(S_\mu) \geq \frac{\mu}{2}$ and $|S_\mu \cap X_i| = k_i$ for both $i \in \{1, 2\}$ after post-processing.

Proof. The candidate S_μ before post-processing has exactly $k = k_1 + k_2$ elements but may not contain k_1 elements from X_1 and k_2 elements from X_2 . If S_μ has exactly k_1 elements from X_1 and k_2 elements from X_2 , and thus the post-processing is skipped, we have $\text{div}(S_\mu) \geq \mu$ according to Theorem 1. Otherwise, assuming that $|S_\mu \cap X_1| = k'_1 < k_1$, we will add $k_1 - k'_1$ elements from $S_{\mu,1}$ to S_μ and remove $k_1 - k'_1$ elements from $S_\mu \cap X_2$ to ensure the fairness constraint. In Line 11, all the k_1 elements in $S_{\mu,1}$ can be selected for insertion. Since the minimum distance between any pair of elements in $S_{\mu,1}$ is at least μ , we can find, at most, one element $x \in S_{\mu,1}$, such that $d(x, y) < \frac{\mu}{2}$ for each $y \in S_\mu \cap X_1$. This means that there are at least $k_1 - k'_1$ elements from $S_{\mu,1}$ whose distances to all the existing elements in $S_\mu \cap X_1$ are greater than $\frac{\mu}{2}$. Accordingly, after adding $k_1 - k'_1$ elements from $S_{\mu,1}$ to S_μ greedily, it still holds that $d(x, y) \geq \frac{\mu}{2}$ for any $x, y \in S_\mu \cap X_1$. In Line 14, for each element $x \in S_\mu \cap X_2$, there is, at most, one (newly added) element $y \in S_\mu \cap X_1$ such that $d(x, y) < \frac{\mu}{2}$. Meanwhile, it is guaranteed that y is the nearest neighbor of x in S_μ in this case. Therefore, in Line 14, every $x \in S_\mu \cap X_2$ with $d(x, S_\mu \cap X_2) < \frac{\mu'}{2}$ is removed, since there are, at most, $k_1 - k'_1$ such elements and the one with the smallest $d(x, S_\mu \cap X_2)$ is removed at each step. Therefore, S_μ contains k_1 elements from X_1 and k_2 elements from X_2 and $\text{div}(S_\mu) \geq \frac{\mu}{2}$ after post-processing. $\square$

Theorem 2. SFDM1 returns a $\frac{1-\varepsilon}{4}$ -approximate solution for FDM.

Proof. According to the results of Lemmas 1 and 2, we have $\text{div}(S) \geq \text{div}(S_{\mu'}) \geq \frac{\mu'}{2} \geq \frac{1-\varepsilon}{4} \cdot \text{OPT}_f$, where $\mu' = \max_{\mu \in \mathcal{U}'} \mu$. $\square$

Theorem 3. SFDM1 stores $O(\frac{k \log \Delta}{\varepsilon})$ elements in memory, takes $O(\frac{k \log \Delta}{\varepsilon})$ time per element for stream processing, and $O(\frac{k^2 \log \Delta}{\varepsilon})$ time for post-processing.

Proof. SFDM1 keeps three candidates for each $\mu \in \mathcal{U}$ and $O(k)$ elements in each candidate. Hence, the total number of stored elements is $O(\frac{k \log \Delta}{\varepsilon})$, since $|\mathcal{U}| = O(\frac{\log \Delta}{\varepsilon})$. The stream processing performs, at most, $O(\frac{k \log \Delta}{\varepsilon})$ distance computations per element. Finally, for each $\mu \in \mathcal{U}'$ in the post-processing, at most $k_i(k_i - k'_i)$ distance computations are performed to select the elements in $S_{\mu,i}$ that are to be added to S_μ . To find the elements that are to be removed, at most $k(k_i - k'_i)$ distance computations are needed. Thus, the time complexity for post-processing is $O(\frac{k^2 \log \Delta}{\varepsilon})$ as $|\mathcal{U}'| = O(\frac{\log \Delta}{\varepsilon})$. $\square$

Comparison with Prior Art: The idea of finding a solution and balancing it for fairness in SFDM1 has also been used for FairSwap [17]. However, FairSwap only works in the offline setting, which keeps the dataset in memory and requires random accesses for computation, whereas SFDM1 works in the streaming setting, which scans the dataset in one pass and uses only the elements in the candidates for post-processing. Compared with FairSwap, SFDM1 reduces the space complexity from $O(n)$ to $O(\frac{k \log \Delta}{\varepsilon})$ and the time complexity from $O(nk)$ to $O(\frac{k^2 \log \Delta}{\varepsilon})$ at the expense of lowering the approximation ratio by a factor of $1 - \varepsilon$.

4.3. Fair Streaming Algorithm for General m

The detailed procedure of our streaming algorithm, which can work with an arbitrary $m \geq 2$, called SFDM2, is presented in Algorithm 3. Similar to SFDM1, it also has two phases: stream processing and post-processing. In the stream processing (Lines 1–7), it utilizes Algorithm 1 to keep a group-blind candidate S_μ and m group-specific candidates $S_{\mu,1}, \dots, S_{\mu,m}$ for all the m groups. The difference from SFDM1 is that the size constraint of each group-specific candidate for each group i is k instead of k_i . Then, after processing all elements in X , a post-processing scheme is required to ensure the fairness of candidates. Nevertheless, the post-processing procedures are totally different from SFDM1, since the swap-based balancing strategy cannot guarantee the validity of the solution with any theoretical bound. Like SFDM1, the post-processing is performed on a subset $\mathcal{U}'$, where S_μ has k elements and $S_{\mu,i}$ has at least k_i elements for each group i (Line 8). For each $\mu \in \mathcal{U}'$, it initializes with a subset S'_μ of S_μ (Line 10). For an over-filled group i , i.e., $|S_\mu \cap X_i| > k_i$, S'_μ contains k_i arbitrary elements from S_μ . For an under-filled or exactly filled group i , i.e., $|S_\mu \cap X_i| \leq k_i$, S'_μ contains all $k'_i = |S_\mu \cap X_i|$ elements from S_μ . Next, new elements from under-filled groups should be added to S'_μ so that S'_μ is a fair solution. The method used to find the elements that are to be added is to divide the set S_{all} of elements in all candidates into a set $\mathcal{C}$ of clusters, which guarantees that $d(x, y) \geq \frac{\mu}{m+1}$ for any $x \in C_a$ and $y \in C_b$ (Lines 12–15), where C_a and C_b are two different clusters in $\mathcal{C}$. Then, S'_μ is limited to contain, at most, one element from each cluster after new elements are added so that $\text{div}(S'_\mu) \geq \frac{\mu}{m+1}$. Meanwhile, S'_μ should still satisfy the fairness constraint. To meet both requirements, the problem of adding new elements to S'_μ is formulated as an instance of matroid intersection [41–43], as will be discussed subsequently (Line 17). Finally, it returns S'_μ containing k elements with maximum diversity after post-processing as the final solution for FDM (Line 18). An illustration of the post-processing procedure of SFDM2 is given in Figure 4.

Algorithm 3 SFDM2

Input: Stream $X = \bigcup_{i=1}^m X_i$, distance metric d , parameter $\varepsilon \in (0, 1)$, size constraints $k_1, \dots, k_m \in \mathbb{Z}^+$ ($k = \sum_{i=1}^m k_i$)

Output: A set $S \subseteq X$ s.t. $|S \cap X_i| = k_i, \forall i \in [m]$

▷ Stream processing

- 1: $\mathcal{U} = \{ \frac{d_{\min}}{(1-\varepsilon)^j} : j \in \mathbb{Z}_0^+ \wedge (1-\varepsilon)^j \geq \frac{d_{\min}}{d_{\max}} \}$
- 2: Initialize $S_\mu, S_{\mu,i} = \emptyset$ for every $\mu \in \mathcal{U}$ and $i \in [m]$
- 3: **for all** $x \in X$ **do**
- 4: **for all** $\mu \in \mathcal{U}$ and $i \in [m]$ **do**
- 5: Run Lines 3–6 of Algorithm 1 to update S_μ w.r.t. x
- 6: **if** $c(x) = i$ **then**
- 7: Run Lines 3–6 of Algorithm 1 to update $S_{\mu,i}$ w.r.t. x
- 8: ▷ Post-processing
- 9: $\mathcal{U}' = \{ \mu \in \mathcal{U} : |S_\mu| = k \wedge |S_{\mu,i}| \geq k_i, \forall i \in [m] \}$
- 10: **for all** $\mu \in \mathcal{U}'$ **do**
- 11: For each group $i \in [m]$, pick $\min(k_i, |S_\mu \cap X_i|)$ elements arbitrarily from S_μ as S'_μ
- 12: Let $S_{all} = (\bigcup_{i=1}^m S_{\mu,i}) \cup S_\mu$ and $l = |S_{all}|$
- 13: Create l clusters $\mathcal{C} = \{C_1, \dots, C_l\}$, each of which contains one element in S_{all}
- 14: **while** there exist $C_a, C_b \in \mathcal{C}$ s.t. $d(x, y) < \frac{\mu}{m+1}$ for some $x \in C_a$ and $y \in C_b$ **do**
- 15: Merge C_a, C_b into a new cluster $C' = C_a \cup C_b$
- 16: $\mathcal{C} \leftarrow \mathcal{C} \setminus \{C_a, C_b\} \cup \{C'\}$
- 17: Let $\mathcal{M}_1 = (S_{all}, \mathcal{I}_1)$ and $\mathcal{M}_2 = (S_{all}, \mathcal{I}_2)$ be two matroids, where $S \in \mathcal{I}_1$ iff $|S \cap X_i| \leq k_i, \forall i \in [m]$ and $S \in \mathcal{I}_2$ iff $|S \cap \mathcal{C}| \leq 1, \forall \mathcal{C} \in \mathcal{C}$
- 18: Run Algorithm 4 to augment S'_μ such that S'_μ is a maximum cardinality set in $\mathcal{I}_1 \cap \mathcal{I}_2$
- 19: **return** $S \leftarrow \arg \max_{\mu \in \mathcal{U}' : |S'_\mu| = k} \text{div}(S'_\mu)$

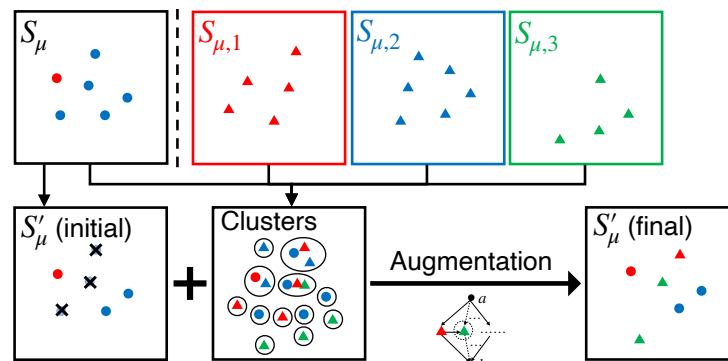


Figure 4. Illustration of post-processing in SFDM2. For each $\mu \in \mathcal{U}'$, an initial S'_μ is first extracted from S_μ by removing the elements from over-filled groups. Then, the elements in all candidates are divided into clusters. The final S'_μ is augmented from the initial solution by adding new elements from under-filled groups based on matroid intersection.

Matroid Intersection: Next, we describe how to use matroid intersection for solution augmentation in SFDM2. We define the first rank- k matroid $\mathcal{M}_1 = (V, \mathcal{I}_1)$ based on the fairness constraint, where the ground set V is S_{all} and $S \in \mathcal{I}_1$ iff $|S \cap X_i| \leq k_i, \forall i \in [m]$. Intuitively, a set S is fair if it is a maximal independent set in $\mathcal{I}_1$. Moreover, we define the second rank- l ($l = |\mathcal{C}|$) matroid $\mathcal{M}_2 = (V, \mathcal{I}_2)$ on the set $\mathcal{C}$ of clusters, where the ground set V is also S_{all} and $S \in \mathcal{I}_2$ if $|S \cap \mathcal{C}| \leq 1, \forall \mathcal{C} \in \mathcal{C}$. Accordingly, the problem of adding new elements to S'_μ to ensure fairness is an instance of the matroid intersection problem, which aims to find a maximum cardinality set $S \in \mathcal{I}_1 \cap \mathcal{I}_2$ for $\mathcal{M}_1 = (S_{all}, \mathcal{I}_1)$ and $\mathcal{M}_2 = (S_{all}, \mathcal{I}_2)$. Here, we adopt Cunningham's algorithm [41], a well-known solution for the matroid intersection problem based on the augmentation graph in Definition 2.

Definition 2 (Augmentation Graph [41]). *Given two matroids $\mathcal{M}_1 = (V, \mathcal{I}_1)$ and $\mathcal{M}_2 = (V, \mathcal{I}_2)$, a set $S \subset V$, such that $S \in \mathcal{I}_1 \cap \mathcal{I}_2$, and two sets $V_1 = \{x \in V \setminus S : S \cup \{x\} \in \mathcal{I}_1\}$ and $V_2 = \{x \in V \setminus S : S \cup \{x\} \in \mathcal{I}_2\}$, an augmentation graph is a digraph $G = (V \cup \{a, b\}, E)$, where $a, b \notin V$. There is an edge $(a, x) \in E$ for each $x \in V_1$. There is an edge $(x, b) \in E$ for each $x \in V_2$. There is an edge $(y, x) \in E$ for each $x \in V \setminus S, y \in S$, such that $S \cup \{x\} \notin \mathcal{I}_1$*

and $S \cup \{x\} \setminus \{y\} \in \mathcal{I}_1$. There is an edge $(x, y) \in E$ for each $x \in V \setminus S$, $y \in S$, such that $S \cup \{x\} \notin \mathcal{I}_2$ and $S \cup \{x\} \setminus \{y\} \in \mathcal{I}_2$.

Specifically, the Cunningham's algorithm [41] is initialized with $S = \emptyset$ (or any $S \in \mathcal{I}_1 \cap \mathcal{I}_2$). At each step, it builds an augmentation graph G for $\mathcal{M}_1$, $\mathcal{M}_2$, and S . If there is no directed path from a to b in G , then S is already a maximum cardinality set. Otherwise, it finds the shortest path P^* from a to b in G , and augments S according to P^* . For each $x \in V \setminus S$, except a and b , add x to S ; for each $x \in S$, remove x from S . We adapt Cunningham's algorithm [41] for our problem, as shown in Algorithm 4. Our algorithm is initialized with S'_μ instead of $\emptyset$. In addition, to reduce the cost of building G and maximize the diversity, it first adds the elements in $V_1 \cap V_2$ greedily to S'_μ until $V_1 \cap V_2 = \emptyset$. This is because a shortest path, $P^* = \langle a, x, b \rangle$ in G , exists for any $x \in V_1 \cap V_2$, which is easy to verify from Definition 2. Finally, if $|S| < k$ after the above procedures, the standard Cunningham's algorithm will be used to augment S to ensure the maximality of S .

Algorithm 4 Matroid Intersection

Input: Two matroids $\mathcal{M}_1 = (V, \mathcal{I}_1)$, $\mathcal{M}_2 = (V, \mathcal{I}_2)$, distance metric d , initial set $S_0 \subseteq V$
Output: A maximum cardinality set $S \subseteq V$ in $\mathcal{I}_1 \cap \mathcal{I}_2$
1: Initialize $S \leftarrow S_0$, $V_1 = \{x \in V \setminus S : S \cup \{x\} \in \mathcal{I}_1\}$, and $V_2 = \{x \in V \setminus S : S \cup \{x\} \in \mathcal{I}_2\}$
2: **while** $V_1 \cap V_2 \neq \emptyset$ **do**
3: $x^* \leftarrow \arg \max_{x \in V_1 \cap V_2} d(x, S)$ and $S \leftarrow S \cup \{x^*\}$
4: **for all** $x \in V_1$ **do**
5: $V_1 \leftarrow V_1 \setminus \{x\}$ if $S \cup \{x\} \notin \mathcal{I}_1$
6: **for all** $x \in V_2$ **do**
7: $V_2 \leftarrow V_2 \setminus \{x\}$ if $S \cup \{x\} \notin \mathcal{I}_2$
8: Build an augmentation graph G for S
9: **while** there is a directed path from a to b in G **do**
10: Let P^* be a shortest path from a to b in G
11: **for all** $x \in P^* \setminus \{a, b\}$ **do**
12: $S \leftarrow S \cup \{x\}$ if $x \notin S$
13: $S \leftarrow S \setminus \{x\}$ if $x \in S$
14: Rebuild G for the updated S
15: **return** S

Theoretical Analysis: We prove that SFDM2 achieves an approximation ratio of $\frac{1-\varepsilon}{3m+2}$ for FDM. The high-level idea of the proof is to connect the clustering procedure in post-processing with the notion of matroid and then to utilize the geometric properties of the clusters and the theoretical results of matroid intersection for approximation. Next, we first show that the set $\mathcal{C}$ of clusters has several important properties (Lemma 3). Then, we prove that Algorithm 4 can return a fair solution for a specific μ based on the properties of $\mathcal{C}$ (Lemma 4). Finally, we analyze the time and space complexities of SFDM2 in Theorem 5.

Lemma 3. The set $\mathcal{C}$ of clusters has the following properties: (i) for any $x \in C_a$ and $y \in C_b$ ($a \neq b$), $d(x, y) \geq \frac{\mu}{m+1}$; (ii) each cluster C contains, at most, one element from S_μ and $S_{\mu,i}$ for any $i \in [m]$; (iii) for any $x, y \in C$, $d(x, y) < \frac{m}{m+1} \cdot \mu$.

Proof. First of all, Property (i) holds from Lines 12–15 of Algorithm 3, since all clusters that do not satisfy it have been merged. Then, we prove Property (ii) by contradiction. Let us construct an undirected graph $G = (V, E)$ for a cluster $C \in \mathcal{C}$, where V is the set of elements in C and there exists an edge $(x, y) \in E$ iff $d(x, y) < \frac{\mu}{m}$. Based on Algorithm 3, for any $x \in C$, there must exist some $y \in C$ ($x \neq y$) such that $d(x, y) < \frac{\mu}{m}$. Therefore, G is a connected graph. Suppose that C contains more than one element from S_μ or $S_{\mu,i}$ for some $i \in [m]$. Let $P_{x,y} = (x, \dots, y)$ be the shortest path of G between x and y , where x and y are both from S_μ or $S_{\mu,i}$. Next, we show that the length of $P_{x,y}$ is, at most, $m + 1$. If the length of $P_{x,y}$ is longer than $m + 1$, there will be a sub-path $P_{x',y'}$ of $P_{x,y}$ where x' and y' are both from S_μ or $S_{\mu,i}$, and this violates the fact that $P_{x,y}$ is the shortest. Since the length of $P_{x,y}$ is, at most, $m + 1$, we have $d(x, y) < (m + 1) \cdot \frac{\mu}{m+1} = \mu$, which contradicts the fact that

$d(x, y) \geq \mu$, as they are both from S_μ or $S_{\mu,i}$. Finally, Property (iii) is a natural extension of Property (ii): since each cluster C contains, at most, one element from S_μ and $S_{\mu,i}$ for any $i \in [m]$, C has, at most, $m + 1$ elements. Therefore, for any two elements $x, y \in C$, the length of the path between them is, at most, m in G and $d(x, y) < m \cdot \frac{\mu}{m+1} = \frac{m}{m+1} \cdot \mu$. $\square$

Lemma 4. If $\text{OPT}_f \geq \frac{3m+2}{m+1} \cdot \mu$, then Algorithm 4 returns a size- k subset S'_μ , such that $S'_\mu \in \mathcal{I}_1 \cap \mathcal{I}_2$ and $\text{div}(S'_\mu) \geq \frac{\mu}{m+1}$.

Proof. First of all, the initial S'_μ is a subset of S_μ . According to Property (ii) of Lemma 3, all elements of S'_μ are in different clusters of $\mathcal{C}$, and thus $S'_\mu \in \mathcal{I}_1 \cap \mathcal{I}_2$. The theoretical results in [41] guarantee that Algorithm 4 can find a size- k set in $\mathcal{I}_1 \cap \mathcal{I}_2$, as long as it exists. Next, we will show such a set exists when $\text{OPT}_f \geq \frac{3m+2}{m+1} \cdot \mu$. To verify this, we need to identify k_i clusters of $\mathcal{C}$ that contain at least one element from X_i for each $i \in [m]$ and show that all $k = \sum_{i=1}^m k_i$ clusters are distinct. Here, we consider two cases for each group $i \in [m]$.

- **Case 1:** For each $i \in [m]$, such that $k_i \leq |S_{\mu,i}| < k$, we have $d(x, S_{\mu,i}) < \mu$ for each $x \in X_i$. Given the optimal solution S_f^* , we define a function f that maps each $x^* \in S_f^*$ to its nearest neighbor in $S_{\mu,i}$. For two elements $x_a^*, x_b^* \in S_f^*$ in these groups, we have $d(x_a^*, f(x_a^*)) < \mu$, $d(x_b^*, f(x_b^*)) < \mu$, and $d(x_a^*, x_b^*) \geq \text{OPT}_f = \text{div}(S_f^*)$. Therefore, $d(f(x_a^*), f(x_b^*)) > \text{OPT}_f - 2\mu$. Since $\text{OPT}_f \geq \frac{3m+2}{m+1} \cdot \mu$, $d(f(x_a^*), f(x_b^*)) > \frac{3m+2}{m+1} \cdot \mu - 2\mu = \frac{m}{m+1} \cdot \mu$. According to Property (iii) of Lemma 3, it is guaranteed that $f(x_a^*)$ and $f(x_b^*)$ are in different clusters. By identifying all the clusters that contain $f(x^*)$ for all $x^* \in S_f^*$, we found k_i clusters for each group $i \in [m]$ such that $k_i \leq |S_{\mu,i}| < k$. All the clusters that were found are guaranteed to be distinct.
- **Case 2:** For all $i \in [m]$ such that $|S_{\mu,i}| = k$, we are able to find k clusters that contain one element from $S_{\mu,i}$ based on Property (ii) of Lemma 3. For such a group i , even though $k - k_i$ clusters have been identified for all other groups, there are still at least k_i clusters available for selection. Therefore, we can always find k_i clusters that are distinct from all the clusters identified by any other group for such a group X_i .

Considering both cases, we have proven the existence of a size- k set in $\mathcal{I}_1 \cap \mathcal{I}_2$. Finally, for any set $S \in \mathcal{I}_2$, we have $\text{div}(S) \geq \frac{\mu}{m+1}$ according to Property (i) of Lemma 3. $\square$

Theorem 4. SFDM2 is a $\frac{1-\epsilon}{3m+2}$ -approximation algorithm for FDM.

Proof. Let μ^- be the smallest $\mu \notin \mathcal{U}'$. It holds that $\mu^- \geq \frac{\text{OPT}_f}{2}$ (see Lemma 1). Thus, there is some $\mu < \mu^-$ in $\mathcal{U}'$, such that $\mu \in [\frac{(m+1)(1-\epsilon)}{3m+2} \cdot \text{OPT}_f, \frac{m+1}{3m+2} \cdot \text{OPT}_f]$, as $\frac{m+1}{3m+2} < \frac{1}{2}$ for any $m \in \mathbb{Z}^+$. Therefore, SFDM2 provides a fair solution S , such that $\text{div}(S) \geq \text{div}(S'_\mu) \geq \frac{\mu}{m+1} \geq \frac{1-\epsilon}{3m+2} \cdot \text{OPT}_f$. $\square$

Theorem 5. SFDM2 keeps $O(\frac{km \log \Delta}{\epsilon})$ elements in memory, takes $O(\frac{k \log \Delta}{\epsilon})$ time per element in the stream processing, and spends $O(\frac{k^2 m \log \Delta}{\epsilon} \cdot (m + \log^2 k))$ time for post-processing.

Proof. SFDM2 keeps $m + 1$ candidates for each $\mu \in \mathcal{U}$ and $O(k)$ elements in each candidate. So, the total number of elements stored by SFDM2 is $O(\frac{km \log \Delta}{\epsilon})$. Only two candidates are checked in streaming processing for each element and thus $O(\frac{k \log \Delta}{\epsilon})$ distance computations are needed. In the post-processing of each μ , we need $O(k)$ time to get the initial solution, $O(k^2 m^2)$ time to cluster S_{all} , and $O(k^2 m)$ time to augment the candidate using Lines 2–7 of Algorithm 4. The time complexity of Cunningham's algorithm is $O(k^2 m \log^2 k)$ according to [42,43]. In sum, the overall time complexity of post-processing is $O(\frac{k^2 m \log \Delta}{\epsilon} \cdot (m + \log^2 k))$. $\square$

Comparison with Prior Art: Existing methods have aimed to find a fair solution based on matroid intersection for fair k -center [21,22,44] and fair max–min diversity maximization [17]. SFDM2 adopts a similar method to FairFlow [17] to construct the clusters and

matroids. However, FairFlow solves matroid intersection as a max-flow problem on a directed graph. Its solution is of poor quality in practice, particularly when m is large. Therefore, SFDM2 uses a different method for matroid intersection based on Cunningham's algorithm, which initializes with a partial solution instead of an empty set for higher efficiency and adds elements greedily like GMM [39] for higher diversity. Hence, SFDM2 has a significantly higher solution quality than FairFlow in practice, though it has a slightly lower approximation ratio.

5. Sliding-Window Algorithms

In this section, we extend our streaming algorithms, i.e., SFDM1 and SFDM2, to the sliding-window model. In Section 5.1, we first present the existing sliding-window algorithm for (unconstrained) diversity maximization [14]. In Section 5.2, we propose our extended sliding-window algorithms for FDM based on the algorithms in Sections 4 and 5.1.

5.1. (Unconstrained) Sliding-Window Algorithm

The unconstrained sliding-window algorithm is shown in Algorithm 5 and illustrated in Figure 5. First of all, it keeps two sequences $\Lambda, \mathcal{U}$, both ranging from d_{min} to d_{max} , to guess the optimum $\text{OPT}[W]$ of DM on the window W (Line 1). For each combination of $\lambda \in \Lambda$ and $\mu \in \mathcal{U}$, it initializes two candidate solutions $A_{\lambda,\mu}$ and $B_{\lambda,\mu}$, each of which will be maintained by Algorithm 1 on two consecutive sub-sequences of X . Two sets $A'_{\lambda,\mu}$ and $B'_{\lambda,\mu}$ to store the replacements of the elements in $A_{\lambda,\mu}$ and $B_{\lambda,\mu}$, in case that they fall out of the sliding window, are also initialized as empty sets (Lines 2 and 3). Then, for each element $x \in X$, it adds x to each $B_{\lambda,\mu}$ using the same method as Algorithm 1. Once x is added to $B_{\lambda,\mu}$, it will be set as its own replacement in $B'_{\lambda,\mu}$ (Lines 7 and 8). Otherwise, it checks whether the distance between x and any existing element in $B_{\lambda,\mu}$ is, at most, μ and assigns x as the replacement of such an element in $B'_{\lambda,\mu}$ (Line 10). Similarly, it also checks whether x can replace any element in $A_{\lambda,\mu}$ and perform the assignment if so (Line 12). After that, if the diversity of any candidate $B_{\lambda,\mu}$ with $|B_{\lambda,\mu}| = k$ exceeds λ , it will remove x from $B_{\lambda,\mu}$, $B'_{\lambda,\mu}$ and set them as $A_{\lambda,\mu}$, $A'_{\lambda,\mu}$, and then re-initialize a new $B_{\lambda,\mu}$ and $B'_{\lambda,\mu}$ with x (Lines 13–16). We describe the post-processing procedure for the window W containing the last w elements in X , which can be easily extended to any window $W^{(T)}$ at time T , in Lines 17–23. It considers two cases for different values of λ, μ : (i) when $A_{\lambda,\mu} \subseteq W$, it runs any algorithm ALG for (centralized) max-min diversity maximization on $A_{\lambda,\mu} \cup B_{\lambda,\mu}$ to find a size- k candidate solution $S_{\lambda,\mu}$ (Line 20); (ii) when $B_{\lambda,\mu} \subseteq W$, ALG is run on $(W \cap A'_{\lambda,\mu}) \cup B_{\lambda,\mu}$, i.e., the non-expired elements from $A'_{\lambda,\mu}$ and $B_{\lambda,\mu}$, instead (Line 22). Finally, the best solution found after post-processing all candidates is returned as the solution S for the window W (Line 23).

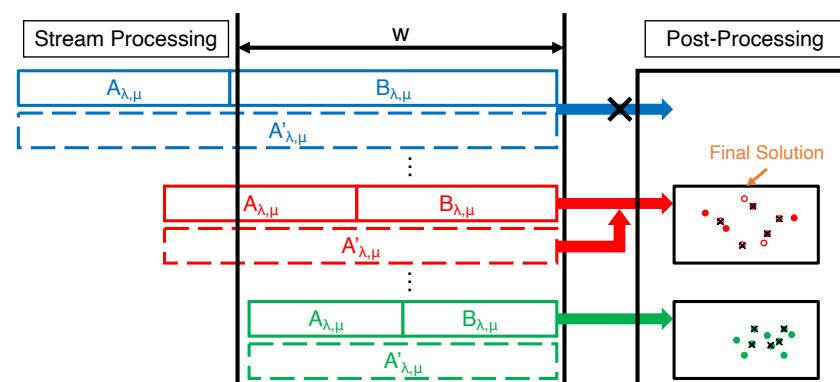


Figure 5. Illustration of the framework of sliding-window algorithms. During stream processing, two candidate solutions $A_{\lambda,\mu}$ and $B_{\lambda,\mu}$, along with their backups $A'_{\lambda,\mu}$ and $B'_{\lambda,\mu}$, are maintained for each guess λ, μ of $\text{OPT}[W]$. Then, during post-processing, the elements in $B_{\lambda,\mu}$ and $A_{\lambda,\mu}$ (or non-expired elements in $A'_{\lambda,\mu}$ if $A_{\lambda,\mu}$ has expired) are passed to an existing algorithm for solution computation.

Algorithm 5 SWDM

Input: Stream X , distance metric $d(\cdot, \cdot)$, window size $w \in \mathbb{Z}^+$, parameter $\varepsilon \in (0, 1)$, solution size $k \in \mathbb{Z}^+$

Output: A set $S \subseteq W$ with $|S| = k$

```

1:  $\Lambda, \mathcal{U} = \{ \frac{d_{\min}}{(1-\varepsilon)^j} : j \in \mathbb{Z}_0^+ \wedge (1-\varepsilon)^j \geq \frac{d_{\min}}{d_{\max}} \}$ 
2: for all  $\lambda \in \Lambda$  and  $\mu \in \mathcal{U}$  do
3:   Initialize  $A_{\lambda, \mu}, A'_{\lambda, \mu} = \emptyset$  and  $B_{\lambda, \mu}, B'_{\lambda, \mu} = \emptyset$ 
4: for all  $x \in X$  do
5:   for all  $\lambda \in \Lambda$  do
6:     for all  $\mu \in \mathcal{U}$  do
7:       if  $|B_{\lambda, \mu}| < k$  and  $d(x, B_{\lambda, \mu}) \geq \mu$  then
8:          $B_{\lambda, \mu} \leftarrow B_{\lambda, \mu} \cup \{x\}, B'_{\lambda, \mu}[x] \leftarrow x$ 
9:       else if  $d(x, B_{\lambda, \mu}) < \mu$  then
10:         $y' \leftarrow \arg \min_{y \in B_{\lambda, \mu}} d(x, y), B'_{\lambda, \mu}[y'] \leftarrow x$ 
11:      if  $A_{\lambda, \mu} \neq \emptyset$  and  $d(x, A_{\lambda, \mu}) < \mu$  then
12:         $y' \leftarrow \arg \min_{y \in A_{\lambda, \mu}} d(x, y), A'_{\lambda, \mu}[y'] \leftarrow x$ 
13:      if  $\max_{\mu \in \mathcal{U}: |B_{\lambda, \mu}|=k} \text{div}(B_{\lambda, \mu}) > \lambda$  then
14:        Remove  $x$  from each  $B_{\lambda, \mu}, B'_{\lambda, \mu}$ 
15:         $A_{\lambda, \mu}, A'_{\lambda, \mu} \leftarrow B_{\lambda, \mu}, B'_{\lambda, \mu}$  for each  $\mu \in \mathcal{U}$ 
16:         $B_{\lambda, \mu}, B'_{\lambda, \mu} \leftarrow \{x\}$  for each  $\mu \in \mathcal{U}$ 
    ▷ Post-processing
17:  $W \leftarrow \{x \in X : \max\{1, |X| - w + 1\} \leq t(x) \leq |X|\}$ 
18: for all  $\lambda \in \Lambda$  and  $\mu \in \mathcal{U}$  do
19:   if  $A_{\lambda, \mu} \subseteq W$  then
20:      $S_{\lambda, \mu} \leftarrow \text{ALG}(k, A_{\lambda, \mu} \cup B_{\lambda, \mu})$ 
21:   else if  $B_{\lambda, \mu} \subseteq W$  then
22:      $S_{\lambda, \mu} \leftarrow \text{ALG}(k, (W \cap A'_{\lambda, \mu}) \cup B_{\lambda, \mu})$ 
23: return  $S \leftarrow \arg \max_{\lambda \in \Lambda, \mu \in \mathcal{U}: |S_{\lambda, \mu}|=k} \text{div}(S_{\lambda, \mu})$ 

```

5.2. Fair Sliding-Window Algorithms

Generally, to extend SFDM1 and SFDM2 so that they can work in the sliding-window model, we need to modify them in two aspects: (i) the stream processing should follow the procedure of Algorithm 5 instead of Algorithm 1 to maintain the candidate solutions for the case when old elements are deleted from the window W ; (ii) the post-processing should be adjusted for the candidate solutions kept by Algorithm 5 during stream processing with theoretical guarantees.

Specifically, the procedures of our extended algorithms, i.e., SWFDM1 and SWFDM2 are presented in Algorithm 6. Here, we put the descriptions of both algorithms together because they share many common subroutines and inherit some others from Algorithms 2–5. Following the procedure of Algorithm 5, they initialize the candidate solutions for different guesses λ, μ of $\text{OPT}[W]$ in the sequences Λ and $\mathcal{U}$. In the stream processing (Lines 1–11), SWFDM1 and SWFDM2 adopts the same method as used in Algorithm 5 to maintain the unconstrained candidate solutions as well as the monochromatic candidate solutions for each group $i \in [m]$. The only difference is the solution size of each monochromatic candidate, which is k_i for $i \in \{1, 2\}$ in SWFDM1 but k for each $i \in [m]$ in SWFDM2.

The following theorem indicates the approximation factor of Algorithm 5.

Theorem 6. Algorithm 5 is a $\frac{\xi-\varepsilon}{5}$ -approximation algorithm for max–min diversity maximization when a ξ -approximation algorithm ALG for (centralized) max–min diversity maximization is used for post-processing.

We refer readers to Lemma 4.7 in [14] for the proof of Theorem 6. Here, if GMM [39], which is $\frac{1}{2}$ -approximate for max–min diversity maximization, is used as ALG, the approximation factor of Algorithm 5 will be $\frac{1-\varepsilon}{10}$. In terms of complexity, Algorithm 5 stores $O(\frac{k \log^2 \Delta}{\varepsilon^2})$ elements, takes $O(\frac{k \log^2 \Delta}{\varepsilon^2})$ time per element for stream processing, and spends $O(\frac{k^2 \log^2 \Delta}{\varepsilon^2})$ time for post-processing.

Algorithm 6 SWFDM

Input: Stream $X = \bigcup_{i=1}^m X_i$, distance metric $d(\cdot, \cdot)$, parameter $\varepsilon \in (0, 1)$, window size $w \in \mathbb{Z}^+$, size constraints $k_1, \dots, k_m$ ($k = \sum_{i=1}^m k_i$)

Output: A set $S \subseteq W$ s.t. $|S \cap X_i| = k_i$ for $i \in [m]$

▷ Stream processing

- 1: $\Lambda, \mathcal{U} = \{ \frac{d_{\min}}{(1-\varepsilon)^j} : j \in \mathbb{Z}_0^+ \wedge (1-\varepsilon)^j \geq \frac{d_{\min}}{d_{\max}} \}$
- 2: **for all** $\lambda \in \Lambda, \mu \in \mathcal{U}$ **do**
- 3: Initialize $A_{\lambda, \mu}, A'_{\lambda, \mu}, B_{\lambda, \mu}, B'_{\lambda, \mu} = \emptyset$
- 4: **for all** $i \in [m]$ **do**
- 5: Initialize $A_{\lambda, \mu}^{(i)}, A'_{\lambda, \mu}{}^{(i)}, B_{\lambda, \mu}^{(i)}, B'_{\lambda, \mu}{}^{(i)} = \emptyset$
- 6: **for all** $x \in X$ **do**
- 7: Run Lines 5–16 of Algorithm 5 to update $A_{\lambda, \mu}, A'_{\lambda, \mu}, B_{\lambda, \mu}$, and $B'_{\lambda, \mu}$ w.r.t. x
- 8: **if** $m = 2 \wedge c(x) = i$ and ‘SWFDM1’ is used **then**
- 9: Run Lines 5–16 of Algorithm 5 to update to update $A_{\lambda, \mu}^{(i)}, A'_{\lambda, \mu}{}^{(i)}, B_{\lambda, \mu}^{(i)}$, and $B'_{\lambda, \mu}{}^{(i)}$ w.r.t. x under size constraint k_i
- 10: **else if** $c(x) = i$ and ‘SWFDM2’ is used **then**
- 11: Run Lines 5–16 of Algorithm 5 to update to update $A_{\lambda, \mu}^{(i)}, A'_{\lambda, \mu}{}^{(i)}, B_{\lambda, \mu}^{(i)}$, and $B'_{\lambda, \mu}{}^{(i)}$ w.r.t. x under size constraint k
- 12: ▷ Post-processing
- 13: **for all** $\lambda \in \Lambda$ and $\mu \in \mathcal{U}$ **do**
- 14: **if** $A_{\lambda, \mu} \subseteq W$ **then**
- 15: $S_{\lambda, \mu} \leftarrow \text{ALG}(k, A_{\lambda, \mu} \cup B_{\lambda, \mu})$
- 16: **else if** $B_{\lambda, \mu} \subseteq W$ **then**
- 17: $S_{\lambda, \mu} \leftarrow \text{ALG}(k, (W \cap A'_{\lambda, \mu}) \cup B_{\lambda, \mu})$
- 18: **if** $m = 2$ and ‘SWFDM1’ is used **then**
- 19: **if** $|S_{\lambda, \mu}| = k \wedge |S_{\lambda, \mu} \cap X_i| < k_i$ **then**
- 20: **if** $A_{\lambda, \mu}^{(i)} \subseteq W$ **then**
- 21: $S_{\lambda, \mu}^{(i)} \leftarrow \text{ALG}(k_i, A_{\lambda, \mu}^{(i)} \cup B_{\lambda, \mu}^{(i)})$
- 22: **else if** $B_{\lambda, \mu}^{(i)} \subseteq W$ **then**
- 23: $S_{\lambda, \mu}^{(i)} \leftarrow \text{ALG}(k_i, (W \cap A'_{\lambda, \mu}{}^{(i)}) \cup B_{\lambda, \mu}^{(i)})$
- 24: Run Lines 10–15 of Algorithm 2 using $S_{\lambda, \mu}$ and $S_{\lambda, \mu}^{(i)}$ as input to find a fair solution $S_{\lambda, \mu}$
- 25: **else if** ‘SWFDM2’ is used **then**
- 26: **for** $i \in [m]$ **do**
- 27: **if** $A_{\lambda, \mu}^{(i)} \subseteq W$ **then**
- 28: $S_{\lambda, \mu}^{(i)} \leftarrow \text{ALG}(k, A_{\lambda, \mu}^{(i)} \cup B_{\lambda, \mu}^{(i)})$
- 29: **else if** $B_{\lambda, \mu}^{(i)} \subseteq W$ **then**
- 30: $S_{\lambda, \mu}^{(i)} \leftarrow \text{ALG}(k, (W \cap A'_{\lambda, \mu}{}^{(i)}) \cup B_{\lambda, \mu}^{(i)})$
- 31: Run Lines 10–17 (with $d(x, y) < \frac{\xi_\mu}{m+1}$ in Line 13) of Algorithm 3 using $S_{\lambda, \mu}$ and $S_{\text{all}} = \bigcup_{i=1}^m S_{\lambda, \mu}^{(i)} \cup S_{\lambda, \mu}$ as input to find a fair solution $S_{\lambda, \mu}$
- 32: **return** $S \leftarrow \arg \max_{\lambda \in \Lambda, \mu \in \mathcal{U} : |S_{\lambda, \mu}| = k} \text{div}(S_{\lambda, \mu})$

The post-processing steps of both algorithms for the window W containing the last w elements in X are shown in Lines 12–31. Note that these steps can be trivially used for any window $W^{(T)}$ based on the intermediate candidate solutions at time T . It first computes an unconstrained solution $S_{\lambda, \mu}$ for each $\lambda \in \Lambda$ and $\mu \in \mathcal{U}$ from the (unconstrained) candidates kept during stream processing based on Algorithm 5. For SWFDM1, it next checks whether $S_{\lambda, \mu}$ has contained k elements and an under-filled group exists in $S_{\lambda, \mu}$. If $|S_{\lambda, \mu}| < k$, the post-processing procedure is skipped because $S_{\lambda, \mu}$ cannot produce any valid solution. Moreover, if $|S_{\lambda, \mu}| = k$ and has already satisfied the fairness constraint, the post-processing will not be required anymore. Otherwise, it computes a group-specific solution $S_{\lambda, \mu}^{(i_u)}$ of size k_{i_u} from the candidates maintained for the under-filled group i_u and performs the procedure as Lines 10–15 of Algorithm 2 to greedily swap the elements from $S_{\lambda, \mu}^{(i_u)}$ into $S_{\lambda, \mu}$ and the elements from the over-filled group i_o out of $S_{\lambda, \mu}$ so that $S_{\lambda, \mu}$ becomes a fair solution. For SWFDM2, it computes a group-specific solution $S_{\lambda, \mu}^{(i)}$ of size k from each group-

specific candidate for $i \in [m]$. $S_{\lambda,\mu}$ as well as each $S_{\lambda,\mu}^{(i)}$ constitutes S_{all} for post-processing. Then, using the same method as Algorithm 3, it picks a subset $S'_{\lambda,\mu}$ of $S_{\lambda,\mu}$, divides S_{all} into clusters, and augments $S'_{\lambda,\mu}$ via matroid intersection as the new solution $S_{\lambda,\mu}$. Both algorithms return the fair solution with maximum diversity after post-processing as the final solution for FDM on the window W .

Theoretical Analysis: Subsequently, we will analyze the theoretical soundness and complexities of the extended SWFDM1 and SWFDM2 algorithms for FDM in the sliding-window model by generalizing the analyses for SFDM1 and SFDM2 in Section 4.

Theorem 7. SWFDM1 is a $\frac{\xi-\varepsilon}{10}$ -approximation algorithm for FDM in the sliding-window model when a ξ -approximation algorithm is used for post-processing. It keeps $O(\frac{k \log^2 \Delta}{\varepsilon^2})$ elements, takes $O(\frac{k \log^2 \Delta}{\varepsilon^2})$ time per element in streaming processing and $O(\frac{k^2 \log^2 \Delta}{\varepsilon^2})$ time for post-processing.

Proof. First, based on the analyses in [14], when $\mu \leq \frac{\text{OPT}[W]}{5}$, there exists $\lambda' \in \Lambda$ such that $\text{div}(S_{\lambda',\mu}) \geq \xi\mu$. Let μ' be the value in $\mathcal{U}$ such that $\mu' \in [(1-\varepsilon) \cdot \frac{\text{OPT}_f[W]}{5}, \frac{\text{OPT}_f[W]}{5}]$, where $\text{OPT}_f[W]$ is the optimal diversity for FDM on window W . Obviously, $\text{OPT}_f[W] \leq \text{OPT}[W]$. Accordingly, we can find the values $\lambda' \in \Lambda$ and $\mu' \in \mathcal{U}$ with $\text{div}(S_{\lambda',\mu'}) \geq \xi\mu'$. Then, Lemma 2 guarantees that $\text{div}(S_{\lambda',\mu'}) \geq \frac{\xi\mu'}{2}$ after the post-processing procedure. Combining the above results, we have $\text{div}(S) \geq \text{div}(S_{\lambda',\mu'}) \geq \frac{(1-\varepsilon)\xi}{10} \cdot \text{OPT}_f[W]$, where S is the solution for FDM on W returned by SWFDM1. Finally, since the number of candidates increases from $O(\frac{\log \Delta}{\varepsilon})$ to $O(\frac{\log^2 \Delta}{\varepsilon^2})$ and the complexities of the remaining steps are not changed, the time and space complexities of SWFDM1 grow by a factor of $\frac{\log \Delta}{\varepsilon}$ compared with SFDM1. $\square$

Theorem 8. SWFDM2 is a $\frac{\xi-\varepsilon}{15m+10}$ -approximation algorithm for FDM in the sliding-window model when a ξ -approximation algorithm is used for post-processing. It keeps $O(\frac{km \log^2 \Delta}{\varepsilon^2})$ elements in memory, and takes $O(\frac{k \log^2 \Delta}{\varepsilon^2})$ time per element in streaming processing and $O(\frac{k^2 m \log^2 \Delta}{\varepsilon^2} \cdot (m + \log^2 k))$ time for post-processing.

Proof. Similar to the proof of Theorem 7, we find the values $\lambda' \in \Lambda$ and $\mu' \in \mathcal{U}$ such that $\mu' \in [(1-\varepsilon) \cdot \frac{\text{OPT}_f[W]}{5}, \frac{\text{OPT}_f[W]}{5}]$ and $\text{div}(S_{\lambda',\mu'}) \geq \xi\mu'$, where $\text{OPT}_f[W]$ is the optimal diversity value for FDM on W . Then, Lemmas 3 and 4 guarantee that $\text{div}(S_{\lambda',\mu'}) \geq \frac{\xi\mu'}{3m+2}$ after the post-processing procedure. Combining the above results, we have $\text{div}(S) \geq \text{div}(S_{\lambda',\mu'}) \geq \frac{(1-\varepsilon)\xi}{15m+10} \cdot \text{OPT}_f[W]$, where S is the solution for FDM on W returned by SWFDM2. Since the number of candidates increases from $O(\frac{\log \Delta}{\varepsilon})$ to $O(\frac{\log^2 \Delta}{\varepsilon^2})$ and the complexities of the remaining steps are not changed, the time and space complexities of SWFDM2 grows by a factor of $\frac{\log \Delta}{\varepsilon}$ compared with SFDM2. $\square$

Finally, since the approximation factor ξ of the algorithm ALG we use is $\Theta(1)$, e.g., $\xi = \frac{1}{2}$ for GMM [39], the approximation factors of SWFDM1 and SWFDM2 are written as $\Theta(1)$ and $\Theta(m^{-1})$, respectively, for simplicity.

6. Experiments

In this section, we evaluate the performance of our proposed algorithms on several real-world and synthetic datasets. We first introduce our experimental setup in Section 6.1. Then, experimental results in the streaming setting are presented in Section 6.2. Finally, experimental results in the sliding-window setting are presented in Section 6.3.

6.1. Experimental Setup

Datasets: Our experiments are conducted on four publicly available real-world datasets, as follows:

- **Adult** (<https://archive.ics.uci.edu/dataset/2/adult>, accessed on 12 July 2023) is a collection of 48,842 records from the 1994 US Census database. We select six numeric attributes as features and normalize each of them to have zero mean and unit standard deviation. The Euclidean distance is used as the distance metric. The groups are generated from two demographic attributes: sex and race. By using them individually and in combination, there are two (sex), five (race), and ten (sex + race) groups, respectively.
- **CelebA** (<https://mmlab.ie.cuhk.edu.hk/projects/CelebA.html>, accessed on 12 July 2023) is a set of 202,599 images of human faces. We use 41 pre-trained class labels as features and the Manhattan distance as the distance metric. We generate two groups from sex {'female', 'male'}, two groups from age {'young', 'not young'}, and four groups from their combination, respectively.
- **Census** (<https://archive.ics.uci.edu/dataset/116/us+census+data+1990>, accessed on 12 July 2023) is a set of 2,426,116 records from the 1990 US Census data. We take 25 (normalized) numeric attributes as features and use the Manhattan distance as the distance metric. We generate 2, 7, and 14 groups from sex, age, and both of them, respectively.
- **Lyrics** (<http://millionsongdataset.com/musixmatch>, accessed on 12 July 2023) is a set of 122,448 documents, each of which is the lyrics of a song. We train a topic model with 50 topics using LDA [45] implemented in Gensim (<https://radimrehurek.com/gensim>, accessed on 12 July 2023). Each document is represented as a 50-dimensional vector and the angular distance is used as the distance metric. We generate 15 groups based on the primary genres of songs.

We also generate different synthetic datasets with varying n and m for scalability tests. In each synthetic dataset, we generate ten two-dimensional Gaussian isotropic blobs with random centers in $[-10, 10]^2$ and identity covariance matrices. We assign points to groups uniformly at random. The Euclidean distance is used as the distance metric. The number n of points varies from 10^3 to 10^7 with fixed $m = 2$ or 10. The number m of groups varies from 2 to 20 with fixed $n = 10^5$. The statistics of all datasets are summarized in Table 1.

Table 1. Statistics of datasets used in the experiments.

Dataset	Group	n	m	# Features	Distance Function
Adult	Sex	48,842	2	6	Euclidean
	Race		5		
	S + R		10		
CelebA	Sex	202,599	2	41	Manhattan
	Age		2		
	S + A		4		
Census	Sex	2,426,116	2	25	Manhattan
	Age		7		
	S + A		14		
Lyrics	Genre	122,448	15	50	Angular
Synthetic	-	10^3 – 10^7	2–20	2	Euclidean

Algorithms: We compare our streaming algorithms, i.e., SFDM1 and SFDM2, and sliding-window algorithms, i.e., SWFDM1 and SWFDM2, with four existing offline FDM algorithms: the $\frac{1}{3m-1}$ -approximation FairFlow algorithm for an arbitrary m , the $\frac{1}{5}$ -approximation FairGMM algorithm for small k and m , and the $\frac{1}{4}$ -approximation FairSwap algorithm specific for $m = 2$ in [17], and the $\frac{1-\epsilon}{m+1}$ -approximation FairGreedyFlow algorithm for an arbitrary m .

in [20]. Since no implementation for the algorithms in [17,20] is available, we implement them by ourselves, following the description in the original paper. All the algorithms are implemented in Python 3. All experiments were run on a desktop with an Intel® Core™ i5-9500 3.0 GHz processor and 32 GB RAM running Ubuntu 20.04.3 LTS. Each algorithm was run on a single thread.

For a given solution size k , the group-specific size constraint k_i for each group $i \in [m]$ is set based on equal representation, which has been widely used in the literature [21–23,27]: If k is divisible by m , $k_i = \frac{k}{m}$ for each $i \in [m]$. If k is not divisible by m , $k_i = \lceil \frac{k}{m} \rceil$ for some groups or $\lfloor \frac{k}{m} \rfloor$ for the others while ensuring $\sum_{i=1}^m k_i = k$. We also compare the performance of different algorithms for proportional representation [8,26,27], another popular notion of fairness that requires a proportion of elements from each group in the solution that are generally preserved in the dataset.

Performance Metrics: The performance of each algorithm is evaluated in terms of efficiency, quality, and space usage. The efficiency is measured as average update time, i.e., the average wall-clock time used to compute a solution for each arrival element in the stream. The quality is measured by the value of the diversity function of the solution returned by an algorithm. Since computing the optimal diversity OPT_f of FDM is infeasible, we run GMM [39] for unconstrained diversity maximization to estimate an upper bound of OPT_f for comparison. Space usage is measured by the number of distinct elements stored by each algorithm. However, only the numbers of elements stored by our proposed algorithms are presented because offline algorithms should keep all elements in memory for random access, and thus their space usage is always equal to the dataset (or window) size. We run each experiment 10 times with different permutations of the same dataset and report the average of each measure over 10 runs.

6.2. Results in Streaming Setting

Effect of Parameter ε : Figure 6 illustrates the performance of SFDM1 and SFDM2 with different values of ε when $k = 20$. We range the value of ε from 0.05 to 0.25 on *Adult*, *CelebA*, and *Census* and from 0.02 to 0.1 on *Lyrics*. Since the angular distances between any two vectors are at most $\frac{\pi}{2}$, larger values of ε (e.g., >0.1) leads to greater estimation errors for OPT_f and thus significantly lower solution quality in *Lyrics*. Generally, SFDM1 has higher efficiency and smaller space usage than SFDM2 for different values of ε , but SFDM2 exhibits a better solution quality. Furthermore, the running time and numbers of stored elements of both algorithms significantly decrease when the value of ε increases. This is consistent with our analyses in Section 4 because the number of guesses for OPT_f , and thus the number of candidates maintained by both algorithms is $O(\frac{\log \Delta}{\varepsilon})$. A slightly surprising result is that the diversity values of the solutions do not obviously degrade, even when $\varepsilon = 0.25$. This can be explained by the fact that both algorithms return the best solutions after post-processing among all candidates, which means that they provide good solutions as long as there is some $\mu \in \mathcal{U}$ close to OPT_f . We infer that μ still exists when $\varepsilon = 0.25$. Nevertheless, we note that the chance of finding an appropriate value of μ will be smaller when the value of ε is larger, which will result in a less stable solution quality. Therefore, in the experiments for streaming setting, we always use $\varepsilon = 0.1$ for both algorithms on all datasets, except *Lyrics*, where the value of ε is set to 0.05. The impact of ε on the performance of SWFDM1 and SWFDM2 is generally similar to that of SFDM1 and SFDM2. However, since the number of candidate solutions is quadratic with respect to ε , we use a larger $\varepsilon = 0.25$ for SWFDM1 and SWFDM2 on all datasets except *Lyrics*, where the value of ε is set to 0.1.

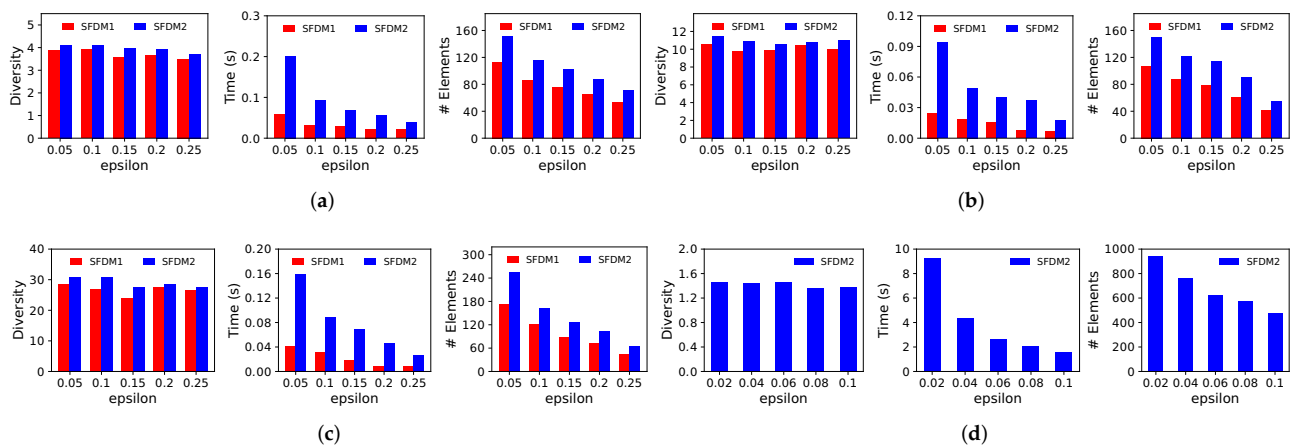


Figure 6. Performance of SFDM1 and SFDM2 with varying parameter ϵ on (a) Adult (Sex, $m = 2$), (b) CelebA (Sex, $m = 2$), (c) Census (Sex, $m = 2$), and (d) Lyrics (Genre, $m = 15$) when $k = 20$.

Overview: Table 2 presents the performance of different algorithms for FDM in the streaming setting on four real-world datasets with different group partitions when the solution size k is fixed to 20. FairGMM is not included because it needs to enumerate, at most, $\binom{km}{k} = O((em)^k)$ candidates for solution computation and cannot scale to $k > 10$ and $m > 5$. First, compared with the unconstrained solution returned by GMM, all the fair solutions are less diverse because of the additional fairness constraints. Since GMM is a $\frac{1}{2}$ -approximation algorithm and $\text{OPT} \geq \text{OPT}_f, 2 \cdot \text{div_GMM}$ is the upper bound of OPT_f , from which we observe that all five fair algorithms return solutions with much better approximation ratios than their lower bounds. In case of $m = 2$, SFDM1 runs the fastest among all five algorithms, which achieves a speed-up of from two to four orders of magnitude over FairSwap, FairFlow and FairGreedyFlow. At the same time, its solution quality is close to or equal to that of FairSwap in most cases. SFDM2 shows lower efficiency than SFDM1 due to the higher cost of post-processing. However, it is still much more efficient than offline algorithms by taking advantage of stream processing. In addition, the solution quality of SFDM2 benefits from the greedy selection procedure in Algorithm 4, which is not only consistently better than that of SFDM1 but also better than that of FairSwap on the *Adult* and *Census* datasets. In the case of $m > 2$, SFDM1 and FairSwap are not applicable anymore. In addition, FairGreedyFlow cannot finish within one day on the *Census* dataset, and the corresponding results are also ignored. SFDM2 shows significant advantages compared to FairFlow and FairGreedyFlow in terms of both solution quality and efficiency. It provides up to 3.4 times more diverse solutions than FairFlow and FairGreedyFlow while running several orders of magnitude faster. In terms of space usage, both SFDM1 and SFDM2 store very small portions of elements ($< 0.1\%$ on *Census*) on all datasets. SFDM2 contains slightly more elements than SFDM1 because the capacity of each group-specific candidate for group i is set to k instead of k_i . For SFDM2, the number of stored elements increases nearly linearly with m since the number of candidates is linear to m .

Table 2. Overview of the performance of different algorithms in the streaming setting ($k = 20$).

Dataset	Group	GMM	FairSwap		FairFlow		FairGreedyFlow		SFDM1			SFDM2		
		Diversity	Diversity	Time (s)	Diversity	Time (s)	Diversity	Time (s)	Diversity	Time (s)	#Elem	Diversity	Time (s)	#Elem
Adult	Sex	5.0226	4.1485	7.06	3.1190	5.45	2.1315	59.99	3.9427	0.0256	90.2	4.1710	0.0965	120.4
	Race		-	-	1.3702	5.82	1.1681	58.59	-	-	-	3.1373	1.0175	312.3
	S + R		-	-	1.0049	6.55	0.8490	60.92	-	-	-	2.9182	3.0914	620.6
CelebA	Sex	13.0	11.4	35.13	8.4	22.97	5.0	705.4	9.8	0.0188	87.2	10.9	0.0410	122.3
	Age		11.4	31.69	7.2	23.06	5.0	657.4	10.4	0.0225	94.6	10.8	0.0591	128.0
	S + A		-	-	6.3	24.17	3.5	312.8	-	-	-	10.4	0.1124	193.1
Census	Sex	35.0	27.0	372.3	17.5	254.6	-	-	27.0	0.0321	121.5	31.0	0.0931	163.0
	Age		-	-	8.5	294.0	-	-	-	-	-	21.0	0.8671	676.0
	S + A		-	-	5.0	347.5	-	-	-	-	-	19.0	3.7539	1276.0
Lyrics	Genre	1.5476	-	-	0.4244	14.84	0.6732	302.6	-	-	-	1.4463	2.6785	677.2

Effect of Solution Size k : The impact of solution size k on the performance of different algorithms in the streaming setting is shown in Figures 7 and 8. Here, we vary k in $[5, 50]$ when $m \leq 5$, or $[10, 50]$ when $5 < m \leq 10$, or $[15, 50]$ when $m > 10$, since we restrict that an algorithm must pick at least one element from each group. For each algorithm, the diversity value drops with k as the diversity function is monotonically non-increasing. At the same time, the update time grows with k , as their time complexities are linear or quadratic w.r.t. k . Compared with the solutions of GMM, all fair solutions are slightly less diverse when $m = 2$. The gaps in diversity values become more apparent when m is larger. Although FairGMM achieves a slightly higher solution quality than any other algorithm when $k \leq 10$ and $m = 2$, it is not scalable to a larger k and m due to the enormous cost of enumeration. The solution quality of FairSwap, SFDM1, and SFDM2 is close to each other when $m = 2$, which is better than that of FairFlow and FairGreedyFlow. However, the efficiencies of SFDM1 and SFDM2 are orders of magnitude higher than those of offline algorithms. Furthermore, when $m > 2$, SFDM2 outperforms FairFlow and FairGreedyFlow in terms of both efficiency and effectiveness across all k values. However, since the time complexity of SFDM2 is quadratic w.r.t. both k and m , its update time increases drastically with k and might be close to that of FairFlow when k and m are large.

Scalability: We evaluate the scalability of each algorithm in the streaming setting on synthetic datasets by varying the dataset size n from 10^3 to 10^7 and the number of groups m from 2 to 20. The results regarding solution quality and update time for different values of n and m when $k = 20$ are presented in Figure 9. First of all, SFDM2 shows a much better scalability than FairFlow and FairGreedyFlow w.r.t. m in terms of solution quality. The diversity value of the solution of SFDM2 only slightly decreases with m and is up to 3 times higher than that of FairFlow and FairGreedyFlow when $m > 10$. However, its update time increases more rapidly with m due to the quadratic dependence on m . Moreover, the diversity values of different algorithms slightly grow with n but are always close to each other for different values of n when $m = 2$. Finally, the running time of offline algorithms is linear to n . However, the update time of SFDM1 and SFDM2 are almost independent of n , as analyzed in Section 4.

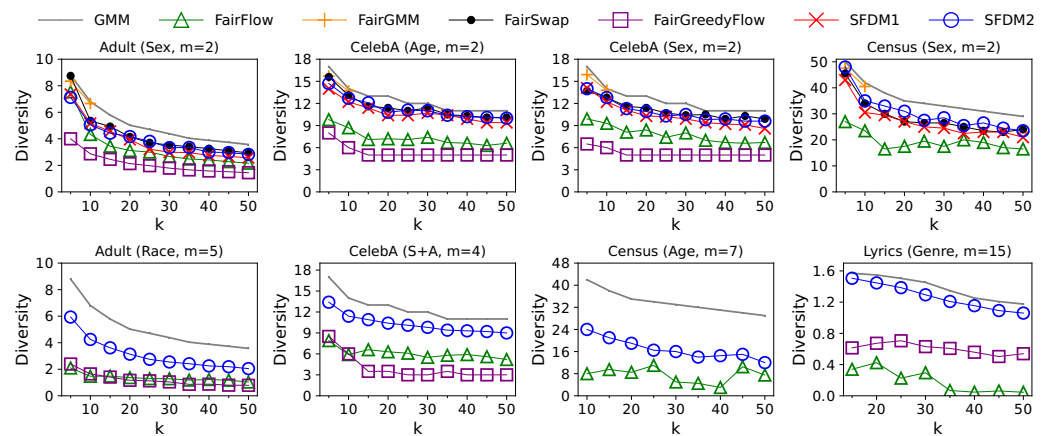


Figure 7. Solution quality of different algorithms in the streaming setting with varying solution sizes, k . The diversity values of GMM are plotted as gray lines to illustrate the “price of fairness”, i.e., the losses in diversity caused by incorporating the fairness constraints.

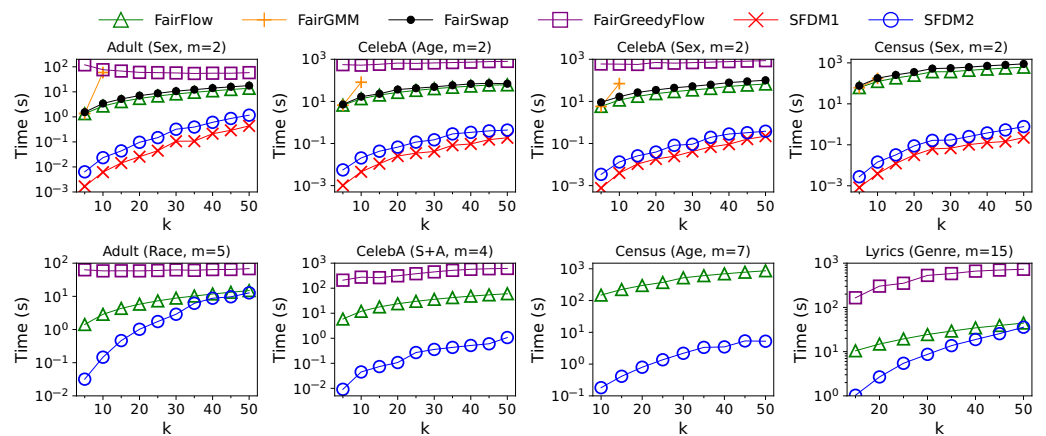


Figure 8. Update time of different algorithms in the streaming setting with varying solution sizes, k .

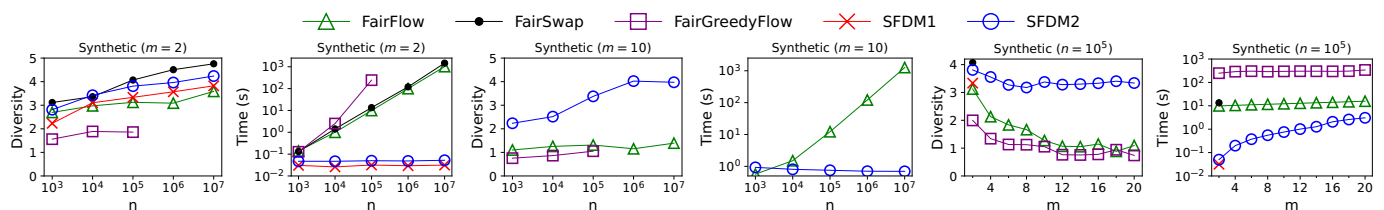


Figure 9. Solution quality and update time on synthetic datasets in the streaming setting with varying dataset sizes, n , and numbers of groups, m ($k = 20$).

Equal vs. Proportional Representation: Figure 10 compares the solution quality and running time of different algorithms for two popular notions of fairness, i.e., equal representation (ER) and proportional representation (PR), when $k = 20$ on *Adult* with highly skewed groups, where 67% of the records are for males and 87% of the records are for Whites. The diversity value of the solution of each algorithm is slightly higher for PR than ER, as the solution for PR is closer to the unconstrained one. The running time of SFDM1 and SFDM2 is slightly shorter for PR than ER, since fewer swapping and augmentation steps are performed on each candidate during post-processing. The results for SWFDM1 and SWFDM2 are similar and will be omitted.

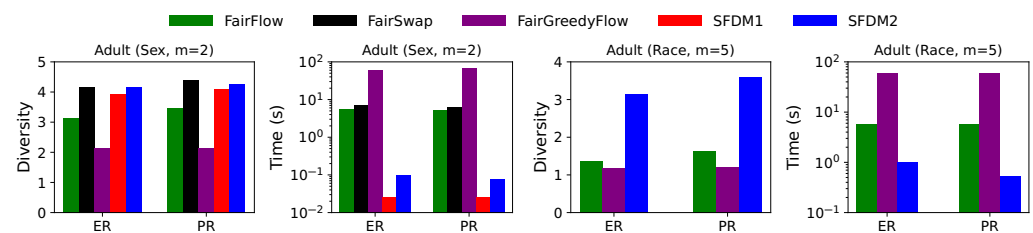


Figure 10. Comparison of different algorithms on *Adult* for equal representation (ER) and proportional representation (PR) when $k = 20$.

6.3. Results in Sliding-Window Setting

Overview: Table 3 shows the performance of different algorithms for sliding-window FDM on four real-world datasets with different group settings when the solution size k is fixed to 20 and the window size w is set to 25 k on *Adult* (as its size is smaller than 100 k) or 100 k on other datasets. FairGMM is also omitted in Table 3 due to its high complexity. Compared with the streaming setting, the “price of fairness” becomes higher in the sliding-window setting, for two possible reasons. First, the approximation factors of our proposed algorithms are lower. Second, some minor groups contain too few elements in the window when the value of m is large (marginally larger than k_i). Thus, the selection of elements from such groups is very restricted to ensure fairness. Nevertheless, we still find that all fair algorithms provide solutions with much better approximations than their lower bounds.

Table 3. Overview of the performance of different algorithms in the sliding-window setting ($k = 20$; $w = 25$ k for *Adult* and $w = 100$ k for other datasets).

Dataset	Group	GMM	FairSwap		FairFlow		FairGreedyFlow		SWFDM1			SWFDM2		
		Diversity	Diversity	Time (s)	Diversity	Time (s)	Diversity	Time (s)	Diversity	Time (s)	#Elem	Diversity	Time (s)	#Elem
Adult	Sex	4.9598	4.0568	6.11	3.0660	5.47	2.0501	13.95	3.5445	0.870	431.5	3.4052	0.489	551.8
	Race		-	-	1.2364	5.87	0.4162	7.71	-	-	-	2.5212	1.056	616.1
	S + R		-	-	0.9105	6.43	0.3276	4.15	-	-	-	1.7843	1.027	799.6
CelebA	Sex	12.0	11.3	28.12	7.7	23.22	6.0	138.4	9.7	2.526	427.7	8.7	0.875	560.6
	Age		10.6	26.99	7.3	23.11	6.0	121.5	10.5	2.966	418.5	8.7	0.976	537.2
	S + A		-	-	6.2	24.10	4.0	112.1	-	-	-	8.8	2.119	864.1
Census	Sex	32.0	30.0	26.33	18.0	21.49	11.0	109.9	29.0	2.593	377.0	28.0	1.614	397.0
	Age		-	-	5.0	22.49	2.0	76.15	-	-	-	13.0	2.568	646.0
	S + A		-	-	2.0	24.11	5.0	128.4	-	-	-	13.0	3.077	796.0
Lyrics	Genre	1.5586	-	-	0.2522	20.12	0.6432	133.4	-	-	-	1.2166	4.694	1132.4

We observe that SWFDM2 runs the fastest of all five algorithms, which achieves 5–150× speedups over FairSwap, FairFlow and FairGreedyFlow. Moreover, SWFDM1 and SWFDM2 have a slightly lower solution quality than FairSwap when $m = 2$. Nevertheless, SWFDM2 shows significant advantages over FairFlow and FairGreedyFlow in terms of both solution quality and efficiency when $m > 2$. Unlike the streaming setting, SWFDM2 shows higher efficiency than SWFDM1. This is because SWFDM2 maintains group-specific solutions with size constraints k , instead of k_i for SWFDM1, in stream processing. Consequently, its group-specific solutions often expire (i.e., $A_{\lambda, \mu}^{(i)} \not\subseteq W$), and thus are not eligible for post-processing. However, such efficiency improvements come at the expense of less diverse solutions. In terms of space usage, both SWFDM1 and SWFDM2 store very small portions of elements (at most $3.2\% \cdot w$) across all datasets. SWFDM2 keeps slightly more elements than SWFDM1 also because the capacity of each group-specific solution is k instead of k_i .

Effect of Solution Size k : The impact of solution size k on the performance of different algorithms in the sliding-window setting is illustrated in Figures 11 and 12. We use the same values of k as in the streaming setting. The window size w is set to $w = 25$ k for *Adult* and 100 k for others. For each algorithm, the diversity value drops with k as the diversity

function is monotonically non-increasing. At the same time, the update time grows with k as their time complexities are linear or quadratic w.r.t. k . The gaps in diversity values between unconstrained and fair solutions are much larger than those in the streaming setting. The reasons for this were explained in the previous paragraph. The solution quality of SWFDM1 and SWFDM2 is slightly lower than FairSwap when $m = 2$, but is still better than that of FairFlow and FairGreedyFlow. However, their efficiencies are always much higher than those of offline algorithms. Finally, when $m > 2$, SWFDM2 outperforms FairFlow and FairGreedyFlow in terms of efficiency and effectiveness across all k values.

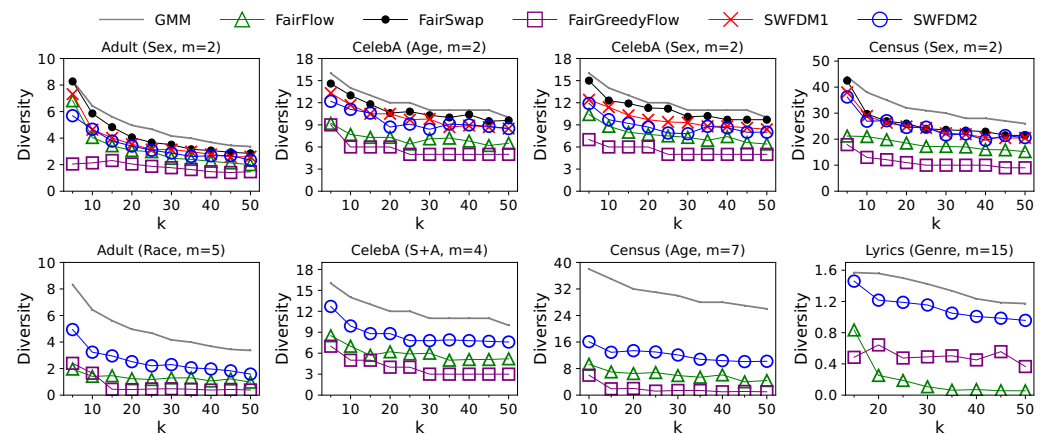


Figure 11. Solution quality of different algorithms in the sliding-window setting with varying solution size k ($w = 25k$ for adult and $100k$ for others). The diversity values of GMM are also plotted as gray lines to illustrate the “price of fairness”.

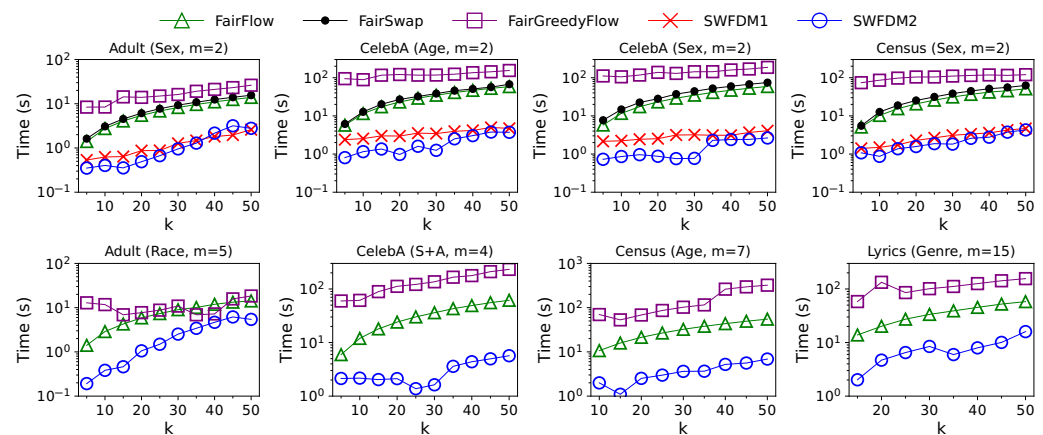


Figure 12. Update time of different algorithms in the sliding-window setting with varying solution size k ($w = 25k$ for Adult and $100k$ for others).

Scalability: We evaluate the scalability of each algorithm in the sliding-window setting on synthetic datasets by varying the number of groups m from 2 to 20 and the window size w from 10^3 to 10^6 . The results regarding solution quality and update time for different values of w and m when $k = 20$ are presented in Figure 13. First of all, SWFDM2 shows a much better scalability than FairFlow and FairGreedyFlow w.r.t. m in terms of solution quality. The diversity value of the solution of SWFDM2 only slightly decreases with m . However, for FairFlow and FairGreedyFlow, the diversity values drop drastically with m . Nevertheless, its update time increases more rapidly with m since its time complexity is quadratic w.r.t. m . Furthermore, the results of the diversity values of different algorithms with varying w are similar to those for varying k . As expected, the running time of offline algorithms is nearly linear to w . However, unlike the streaming setting, the update time of SWFDM1 and SWFDM2 increases with w because more candidates are non-expired and thus considered in post-processing for a larger value of w .

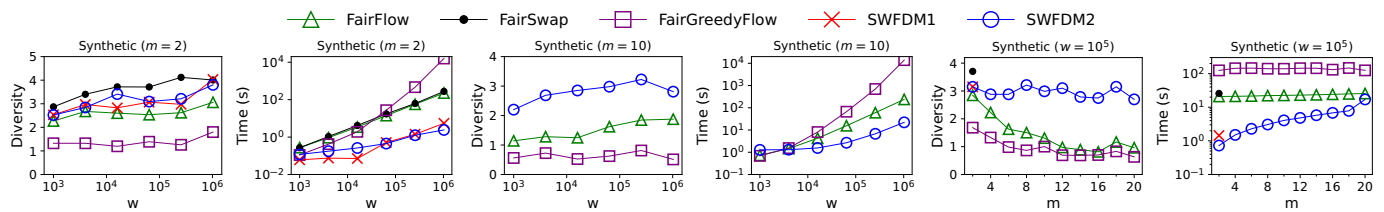


Figure 13. Solution quality and update time on synthetic datasets in the sliding-window setting with varying window size w and number of groups m ($k = 20$).

7. Conclusions

In this paper, we studied the diversity maximization problem with fairness constraints in the streaming and sliding window settings. We first proposed a $\frac{1-\epsilon}{4}$ -approximation streaming algorithm for this problem when there were two groups in the dataset and a $\frac{1-\epsilon}{3m+2}$ -approximation streaming algorithm that could deal with an arbitrary number m of groups. Moreover, we extended the two proposed streaming algorithms to the sliding-window model while maintaining approximation factors of $\Theta(1)$ and $\Theta(m^{-1})$, respectively. Extensive experiments on real-world and synthetic datasets confirmed the efficiency, effectiveness, and scalability of our proposed algorithms.

In future work, we would like to improve the approximation ratios of the proposed algorithms. It would also be interesting to consider diversity maximization problems with other objective functions and fairness constraints defined on multiple sensitive attributes.

Author Contributions: Conceptualization, Y.W. and M.M.; methodology, Y.W.; software, Y.W., F.F. and J.L.; validation, M.M.; data curation, F.F. and J.L.; writing—original draft preparation, Y.W.; writing—review and editing, Y.W., F.F. and M.M.; visualization, Y.W. and J.L.; supervision, Y.W. and M.M.; funding acquisition, Y.W. and M.M. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the MLDB project of the Academy of Finland (decision number: 322046) and the National Natural Science Foundation of China under grant number 62202169.

Institutional Review Board Statement: Not applicable

Data Availability Statement: Real-world datasets that we use in our experiments are publicly available. The code for generating synthetic data and for our experiments is available at <https://github.com/yhwang1990/code-FDM> (accessed on 12 July 2023).

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Ahmed, M. Data summarization: A survey. *Knowl. Inf. Syst.* **2019**, *58*, 249–273. [CrossRef]
2. Qin, L.; Yu, J.X.; Chang, L. Diversifying Top-K Results. *Proc. VLDB Endow.* **2012**, *5*, 1124–1135. [CrossRef]
3. Zheng, K.; Wang, H.; Qi, Z.; Li, J.; Gao, H. A survey of query result diversification. *Knowl. Inf. Syst.* **2017**, *51*, 1–36. [CrossRef]
4. Gollapudi, S.; Sharma, A. An axiomatic approach for result diversification. In Proceedings of the 18th International Conference on World Wide Web, Madrid, Spain, 20–24 April 2009; pp. 381–390.
5. Rafiei, D.; Bharat, K.; Shukla, A. Diversifying web search results. In Proceedings of the 19th International Conference on World Wide Web, Raleigh, NC, USA, 26–30 April 2010; pp. 781–790.
6. Kunaver, M.; Pozrl, T. Diversity in recommender systems—A survey. *Knowl. Based Syst.* **2017**, *123*, 154–162. [CrossRef]
7. Zadeh, S.A.; Ghadiri, M.; Mirrokni, V.S.; Zadimoghaddam, M. Scalable Feature Selection via Distributed Diversity Maximization. In Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence, San Francisco, CA, USA, 4–9 February 2017; pp. 2876–2883.
8. Celis, L.E.; Keswani, V.; Straszak, D.; Deshpande, A.; Kathuria, T.; Vishnoi, N.K. Fair and Diverse DPP-Based Data Summarization. In Proceedings of the 35th International Conference on Machine Learning, Stockholm, Sweden, 10–15 July 2018; pp. 715–724.
9. Borodin, A.; Lee, H.C.; Ye, Y. Max-Sum diversification, monotone submodular functions and dynamic updates. In Proceedings of the 31st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, Scottsdale, AZ, USA, 21–23 May 2012; pp. 155–166.
10. Drosou, M.; Pitoura, E. DisC diversity: Result diversification based on dissimilarity and coverage. *Proc. VLDB Endow.* **2012**, *6*, 13–24. [CrossRef]

11. Abbassi, Z.; Mirrokni, V.S.; Thakur, M. Diversity maximization under matroid constraints. In Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Chicago, IL, USA, 11–14 August 2013; pp. 32–40.
12. Indyk, P.; Mahabadi, S.; Mahdian, M.; Mirrokni, V.S. Composable core-sets for diversity and coverage maximization. In Proceedings of the 33rd ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems, Dallas, TX, USA, 15–17 May 2014; pp. 100–108.
13. Ceccarello, M.; Pietracaprina, A.; Pucci, G. Fast Coreset-based Diversity Maximization under Matroid Constraints. In Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining, Marina Del Rey, CA, USA, 5–9 February 2018; pp. 81–89.
14. Borassi, M.; Epasto, A.; Lattanzi, S.; Vassilvitskii, S.; Zadimoghaddam, M. Better Sliding Window Algorithms to Maximize Subadditive and Diversity Objectives. In Proceedings of the 38th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, Amsterdam, The Netherlands, 1–3 July 2019; pp. 254–268.
15. Bauckhage, C.; Sifa, R.; Wrobel, S. Adiabatic Quantum Computing for Max-Sum Diversification. In Proceedings of the 2020 SIAM International Conference on Data Mining, Cincinnati, OH, USA, 7–9 May 2020; pp. 343–351.
16. Ceccarello, M.; Pietracaprina, A.; Pucci, G.; Upfal, E. MapReduce and Streaming Algorithms for Diversity Maximization in Metric Spaces of Bounded Doubling Dimension. *Proc. VLDB Endow.* **2017**, *10*, 469–480. [[CrossRef](#)]
17. Mounolidou, Z.; McGregor, A.; Meliou, A. Diverse Data Selection under Fairness Constraints. In Proceedings of the 24th International Conference on Database Theory, Nicosia, Cyprus, 23–26 March 2021; pp. 13:1–13:25.
18. Drosou, M.; Pitoura, E. Diverse Set Selection Over Dynamic Data. *IEEE Trans. Knowl. Data Eng.* **2014**, *26*, 1102–1116. [[CrossRef](#)]
19. Zhang, G.; Gionis, A. Maximizing diversity over clustered data. In Proceedings of the 2020 SIAM International Conference on Data Mining, Cincinnati, OH, USA, 7–9 May 2020; pp. 649–657.
20. Addanki, R.; McGregor, A.; Meliou, A.; Mounolidou, Z. Improved Approximation and Scalability for Fair Max-Min Diversification. In Proceedings of the 25th International Conference on Database Theory, Online, 29 March–1 April 2022; pp. 7:1–7:21.
21. Chiplunkar, A.; Kale, S.; Ramamoorthy, S.N. How to Solve Fair k-Center in Massive Data Models. In Proceedings of the 37th International Conference on Machine Learning, Virtual, 13–18 July 2020; pp. 1877–1886.
22. Jones, M.; Nguyen, H.; Nguyen, T. Fair k-Centers via Maximum Matching. In Proceedings of the 37th International Conference on Machine Learning, Virtual, 13–18 July 2020; pp. 4940–4949.
23. Kleindessner, M.; Awasthi, P.; Morgenstern, J. Fair k-Center Clustering for Data Summarization. In Proceedings of the 36th International Conference on Machine Learning, Long Beach, CA, USA, 9–15 June 2019; pp. 3448–3457.
24. Schmidt, M.; Schwiegelshohn, C.; Sohler, C. Fair Coresets and Streaming Algorithms for Fair k-means. In Proceedings of the 17th International Workshop on Approximation and Online Algorithms, Munich, Germany, 12–13 September 2019; pp. 232–251.
25. Huang, L.; Jiang, S.H.; Vishnoi, N.K. Coresets for Clustering with Fairness Constraints. *Adv. Neural Inf. Process. Syst.* **2019**, *32*, 7587–7598.
26. El Halabi, M.; Mitrović, S.; Norouzi-Fard, A.; Tardos, J.; Tarnawski, J.M. Fairness in Streaming Submodular Maximization: Algorithms and Hardness. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 13609–13622.
27. Wang, Y.; Fabbri, F.; Mathioudakis, M. Fair and Representative Subset Selection from Data Streams. In Proceedings of the Web Conference 2021, Ljubljana, Slovenia, 19–23 April 2021; pp. 1340–1350.
28. Wang, Y.; Fabbri, F.; Mathioudakis, M. Streaming Algorithms for Diversity Maximization with Fairness Constraints. In Proceedings of the 38th IEEE International Conference on Data Engineering, Kuala Lumpur, Malaysia, 9–12 May 2022; pp. 41–53.
29. Cevallos, A.; Eisenbrand, F.; Zenklusen, R. Max-Sum Diversity via Convex Programming. In Proceedings of the 32nd International Symposium on Computational Geometry, Boston, MA, USA, 14–18 June 2016; pp. 26:1–26:14.
30. Cevallos, A.; Eisenbrand, F.; Zenklusen, R. Local Search for Max-Sum Diversification. In Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms, Barcelona, Spain, 16–19 January 2017; pp. 130–142.
31. Aghamolaei, S.; Farhadi, M.; Zarrabi-Zadeh, H. Diversity Maximization via Composable Coresets. In Proceedings of the 27th Canadian Conference on Computational Geometry, Kingston, ON, Canada, 10–12 August 2015; pp. 38–48.
32. Chandra, B.; Halldórsson, M.M. Approximation Algorithms for Dispersion Problems. *J. Algorithms* **2001**, *38*, 438–465. [[CrossRef](#)]
33. Erkut, E. The discrete p-dispersion problem. *Eur. J. Oper. Res.* **1990**, *46*, 48–60. [[CrossRef](#)]
34. Ravi, S.S.; Rosenkrantz, D.J.; Tayi, G.K. Heuristic and Special Case Algorithms for Dispersion Problems. *Oper. Res.* **1994**, *42*, 299–310. [[CrossRef](#)]
35. Hassin, R.; Rubinstein, S.; Tamir, A. Approximation algorithms for maximum dispersion. *Oper. Res. Lett.* **1997**, *21*, 133–137. [[CrossRef](#)]
36. Epasto, A.; Mahdian, M.; Mirrokni, V.; Zhong, P. Improved Sliding Window Algorithms for Clustering and Coverage via Bucketing-Based Sketches. In Proceedings of the 2022 ACM-SIAM Symposium on Discrete Algorithms, Virtual, 9–12 January 2022; pp. 3005–3042.
37. Bhaskara, A.; Ghadiri, M.; Mirrokni, V.S.; Svensson, O. Linear Relaxations for Finding Diverse Elements in Metric Spaces. *Adv. Neural Inf. Process. Syst.* **2016**, *29*, 4098–4106.
38. Wang, Y.; Mathioudakis, M.; Li, J.; Fabbri, F. Max-Min Diversification with Fairness Constraints: Exact and Approximation Algorithms. In Proceedings of the 2023 SIAM International Conference on Data Mining (SDM), Minneapolis, MI, USA, 27–29 April 2023; pp. 91–99.
39. Gonzalez, T.F. Clustering to Minimize the Maximum Intercluster Distance. *Theor. Comput. Sci.* **1985**, *38*, 293–306. [[CrossRef](#)]

40. Korte, B.; Vygen, J. *Combinatorial Optimization: Theory and Algorithms*; Springer: Berlin/Heidelberg, Germany, 2012.
41. Cunningham, W.H. Improved Bounds for Matroid Partition and Intersection Algorithms. *SIAM J. Comput.* **1986**, *15*, 948–957. [[CrossRef](#)]
42. Chakrabarty, D.; Lee, Y.T.; Sidford, A.; Singla, S.; Wong, S.C. Faster Matroid Intersection. In Proceedings of the 60th IEEE Annual Symposium on Foundations of Computer Science, Baltimore, MD, USA, 9–12 November 2019; pp. 1146–1168.
43. Nguyen, H.L. A note on Cunningham’s algorithm for matroid intersection. *arXiv* **2019**, arXiv:1904.04129.
44. Chen, D.Z.; Li, J.; Liang, H.; Wang, H. Matroid and Knapsack Center Problems. *Algorithmica* **2016**, *75*, 27–52. [[CrossRef](#)]
45. Blei, D.M.; Ng, A.Y.; Jordan, M.I. Latent Dirichlet Allocation. *J. Mach. Learn. Res.* **2003**, *3*, 993–1022.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.